

软件项目管理复习

第一章：软件项目管理概述

项目的定义：项目就是为了创建一个唯一的产品或提供一个唯一的服务而进行的临时性的努力；是以一套独特而项目联系的任务为前提，有效地利用资源，在一定时间内满足一系列特定目标的多项相关工作的总称。

项目的特征：（简答题）

目标性，相关性，临时性，独特性，资源约束性，不确定性

- 1、项目有明确的目标
- 2、项目之间的活动具有相关性
- 3、项目有限定的周期
- 4、项目具有独特性
- 5、项目具有资源成本的约束性
- 6、项目由不确定性

这边还会考是非题，哪些是项目，哪些不是项目（常识）

软件项目的特殊性：逻辑实体，相互作用的系统，渐近明细，变更

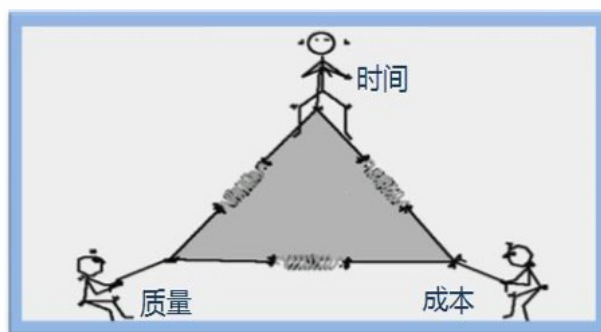


项目管理：一系列的伴随着项目的进行而进行的、目的是为了项目能够达到期望的结果的一系列管理行为。

软件项目管理：

软件工程组成部分。

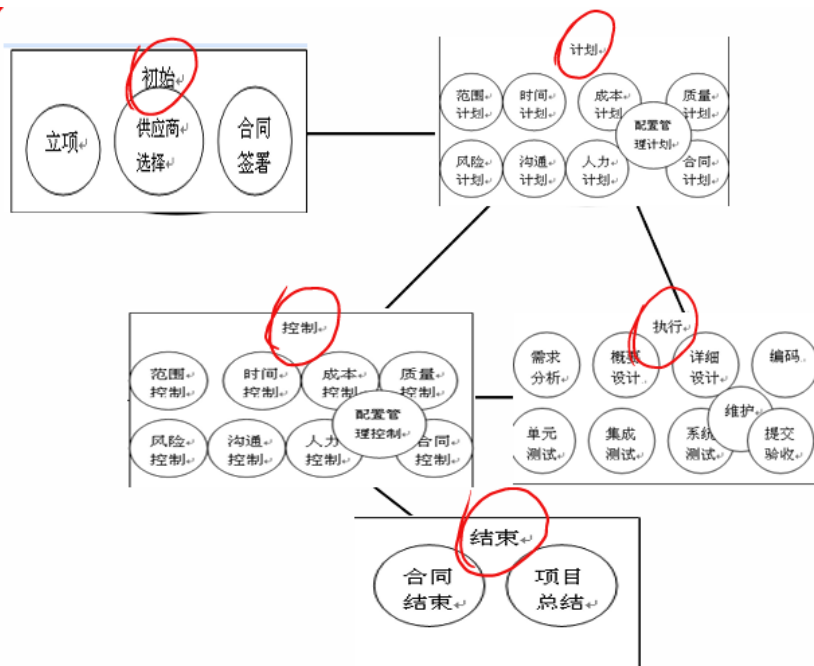
确保软件项目满足预算，成本等约束，提交高质量软件产品。



项目管理
“铁三角”

项目概念里知识体系指南：（PMI）对项目管理所需的知识、技能和工具进行概括性描述

软件项目管理知识体系：初始，计划，控制，执行，结束



敏捷模型：

敏捷组织提出的一个灵活开发方法
 应对迅速变化需求的快速软件开发方法
 是一种迭代、循序渐进的开发方法

敏捷宣言4价值：

个体和互动高于流程和工具。
 可工作的软件高于详尽的文档。
 客户合作高于合同谈判。
 响应变化高于遵循计划。

Principles敏捷原则：

Principles 敏捷原则 判, 选

1. 我们最先要做的是通过尽早地、持续地交付有价值的软件来使客户满意。
2. 即使到了开发的后期，也欢迎改变需求。敏捷过程利用适应变化来为客户创造竞争优势。
3. 经常性地交付可以工作的软件，交付的间隔可以从几个星期到几个月，交付的时间间隔越短越好。
4. 在整个项目开发期间，业务人员和开发人员可以的话尽可能在一起工作。

- 1、我们最先要做的是通过尽早地、持续地交付有价值的软件来使客户满意
- 2、即使到了开发的后期，也欢迎改变需求。敏捷过程利用适应变化来为客户创造竞争优势。
- 3、经常性地交付可以工作的软件、交付的间隔可以从几个星期到几个月，交付的时间间隔越短越好。
- 4、在整个项目开发期间，业务人员和开发人员可以的话尽可能在一起工作。

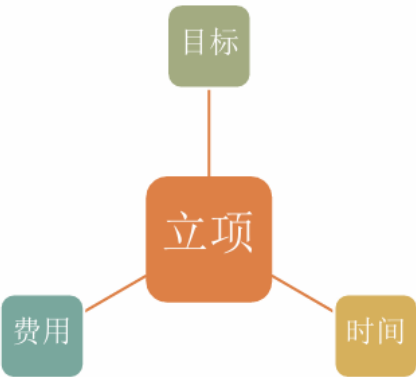
《敏捷宣言》价值观、原则和通用实践之间的关系

敏捷是一种思维模式，它由《敏捷宣言》的价值观所界定，受《敏捷宣言》原则指导，并通过各种实践实现

第二章：项目确立

项目立项：明确项目的目标、时间表、项目使用的资源和经费，而且得到执行该项目的项目经理和项目发起人的认可

明确项目的目标、时间表、项目使用的资源和经费，而且得到执行该项目的项目经理和项目发起人的认可。



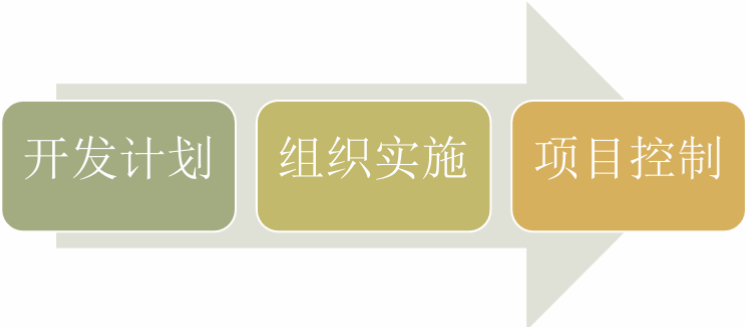
项目招投标过程：甲方招标书定义 乙方项目分析 招标与竞标 合同签署

项目章程：确认项目存在的文件，包括**对项目的确认、对项目经理的授权**和项目目标的概述等

敏捷项目章程基本要素：项目目标，发布标准，预期的工作流

项目经理的职责：

项目经理是领导团队实现项目目标的个人



敏捷强调：仆人式领导方式

第三章：生存期模型

这边直接看书，或者单开一个文档，太多了

第四章：软件项目需求管理

软件需求定义：需求是指用户对软件的功能和性能的要求。

软件需求管理的过程：

需求确认：需求获取，需求分析，需求规格编写，需求验证

需求变更：需求变更

需求分析是为最终用户所看到的系统建立一个概念模型，是对需求的具体抽象描述。

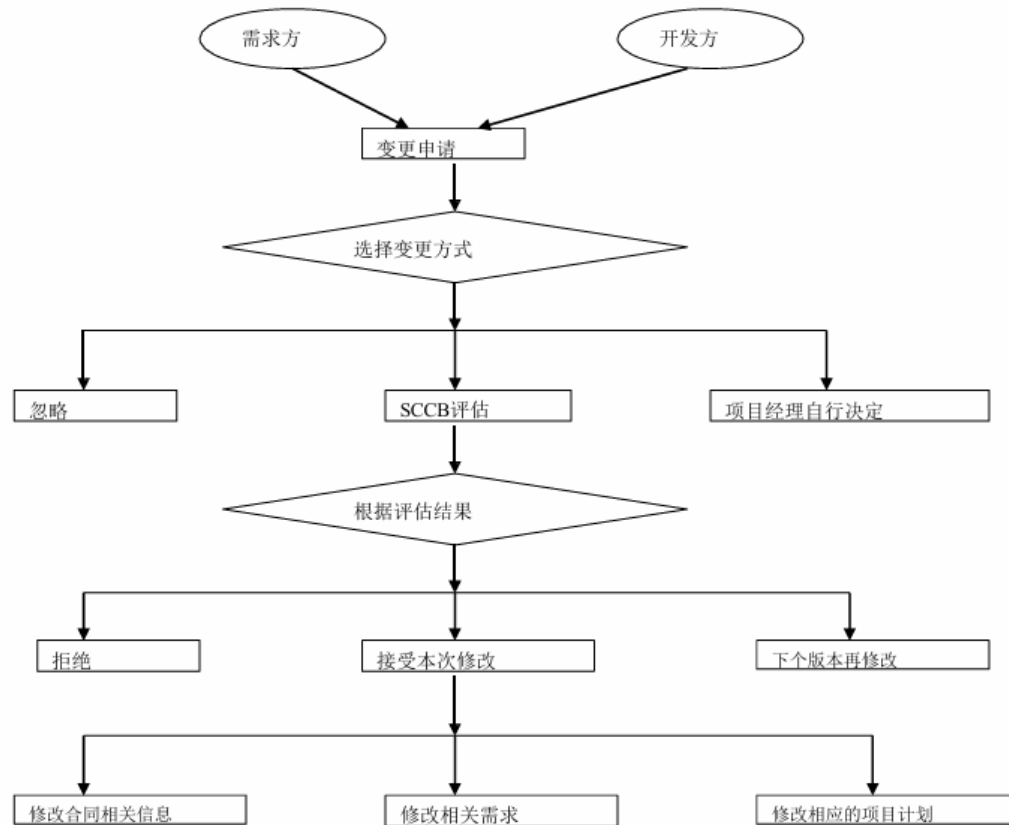
需求规格编写：

需求分析工作完成的一个基本标志是形成了一份完整的、规范的需求规格说明书

需求变更管理：

- 1、确定需求变更控制过程
- 2、建立变更控制委员会（SCCB）
- 3、进行需求变更影响分析
- 4、跟踪所有受需求变更影响的工作产品
- 5、建立需求基准版本和需求控制版本文档
- 6、维护需求变更的历史记录
- 7、跟踪每项需求的状态
- 8、衡量需求稳定性

需求变更控制流程：



需求建模的基本方法介绍：

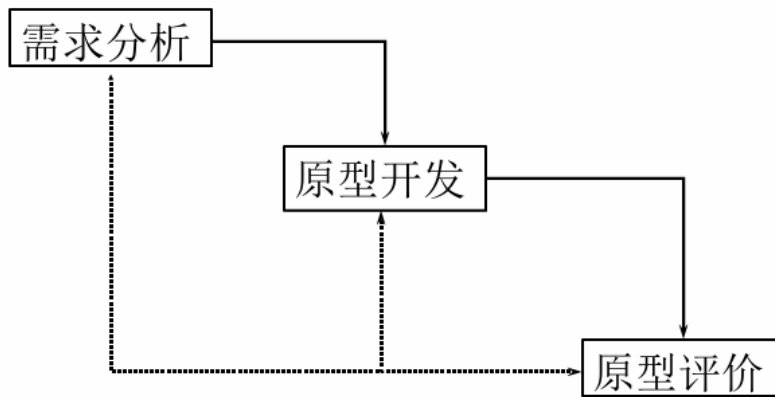
分为传统方法与敏捷方法。

传统方法：

- 1、原型方法
- 2、基于数据流建模
- 3、基于UML建模

原型方法：

敏捷方法



基于数据流-结构化分析方法：

20世纪70年代发展起来的面向数据流的方法。

是一种**自顶向下逐步求精**的分析方法。

根据软件内部**数据传递、变换的关系**进行分析。

基于数据流的技术：

- 1、数据流图（DFD）
- 2、数据字典（DD）
- 3、系统流程图

基于UML方法：

基于面向对象的情景分析方法。

从用户角度出发考虑的功能需求。

用例是系统向用户提供一个有价值的结果的某项功能。

UML需求视图：用例视图，顺序图，状态图，活动图

基于UML方法综述：

识别出系统的Actor

描述需要的Use case

实现用例视图

实现顺序视图，活动视图，状态视图等

Product Backlog:产品待办事项列表

- 需求的来源
- 包含产品想法的一个有序列表
- 一个长短不定列表
- 可以是模糊的或是不具体的
- 逐渐完善，越来越明确

Sprint Backlog:待办事项列表的细化



- ▮ 按照迭代计划,逐步细化需求,形成Story(故事),
- ▮ 鼓励开发人员、测试人员、业务分析人员和产品负责人合作编写故事,
- ▮ 确保所有的故事都足够小,可以持续交付工作。
- ▮ 最好每天完成至少一个故事。

第五章：任务分解

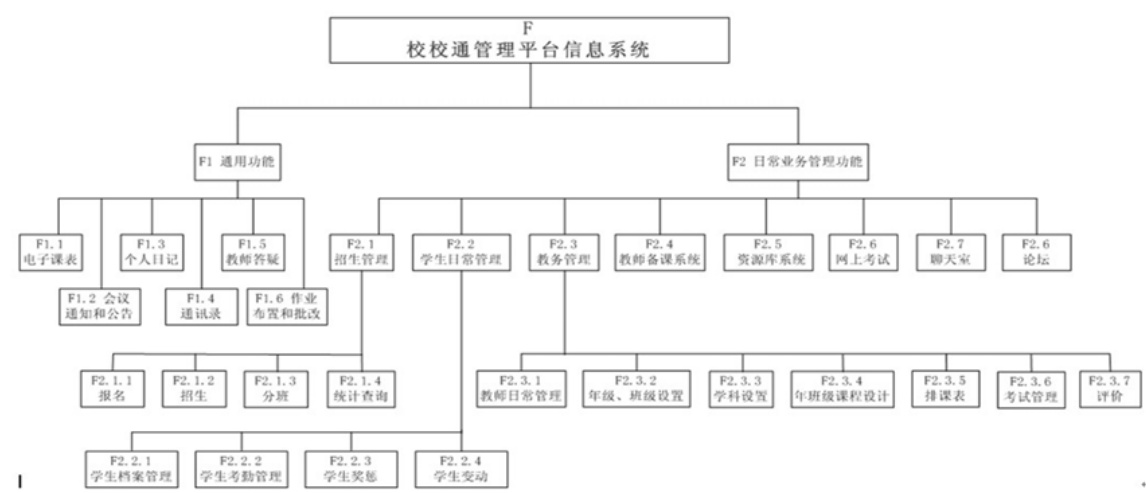
任务分解:

任务分解过程:

将一个项目分解为更多的工作细目或者子项目，使项目变得更小、更易管理、更易操作。

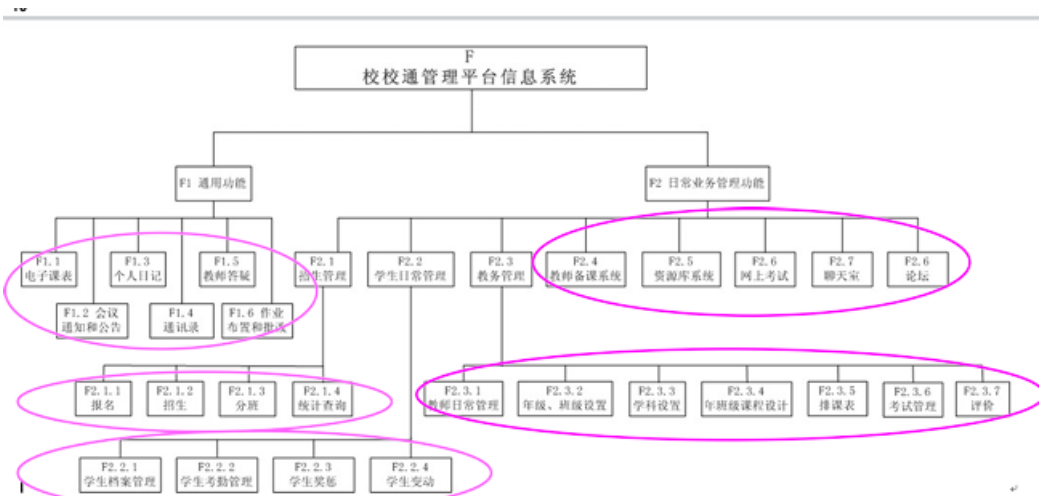
任务分解结果:

WBS(Work Breakdown Structure: 任务分解结构)



- WBS是对项目由粗到细的分解过程。
- 面向交付成果的
- WBS组织并定义了整个项目范围

工作包(Work packages)



- WBS的最低层次的可交付成果
- 工作包应当由唯一主体负责

组织结构图(图表)形式的WBS：



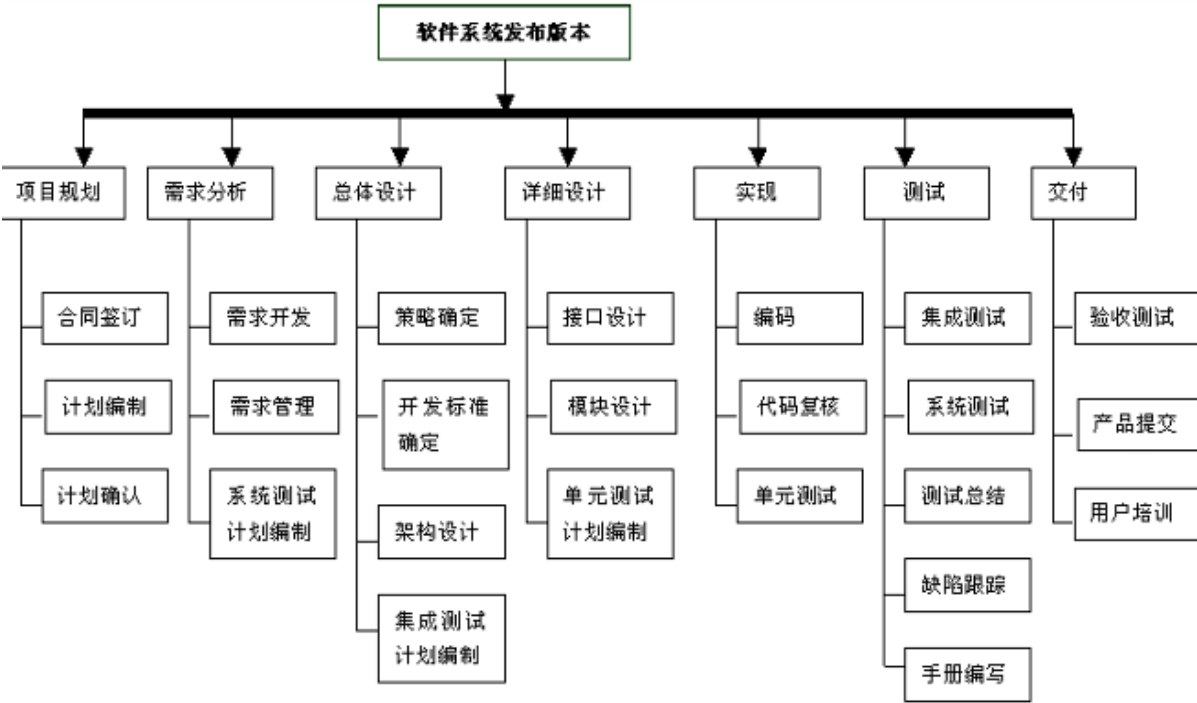
提纲(清单) 式的WBS：

- 1. 变化计数器
 - 1.1 比较两个版本的程序
 - 1.1.1 预处理
 - 1.1.2 文件比较
 - 1.1.3 结果处理
 - 1.2 找出修改后的程序中增加和删除的代码行
 - 1.2.1 找出增加的代码行
 - 1.2.2 找出删除的代码行
 - 1.3 统计修改后的程序中增加和删除的代码行数
 - 1.3.1 统计增加代码行数
 - 1.3.2 统计删除代码行数
 - 1.4 统计总的代码行数
 - 1.5 设定标记以指示修改的次数
 - 1.6 在程序的头部增加修改记录

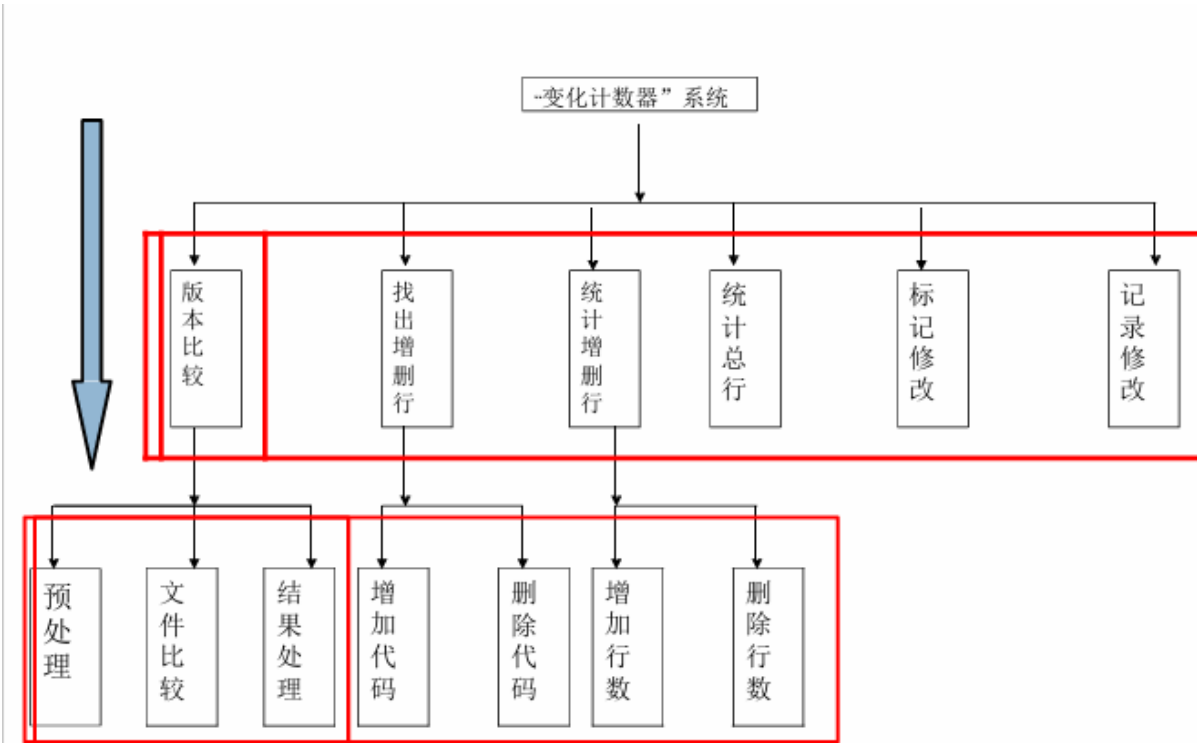
任务分解方法：

类比、模板参照、自上而下、自下而上

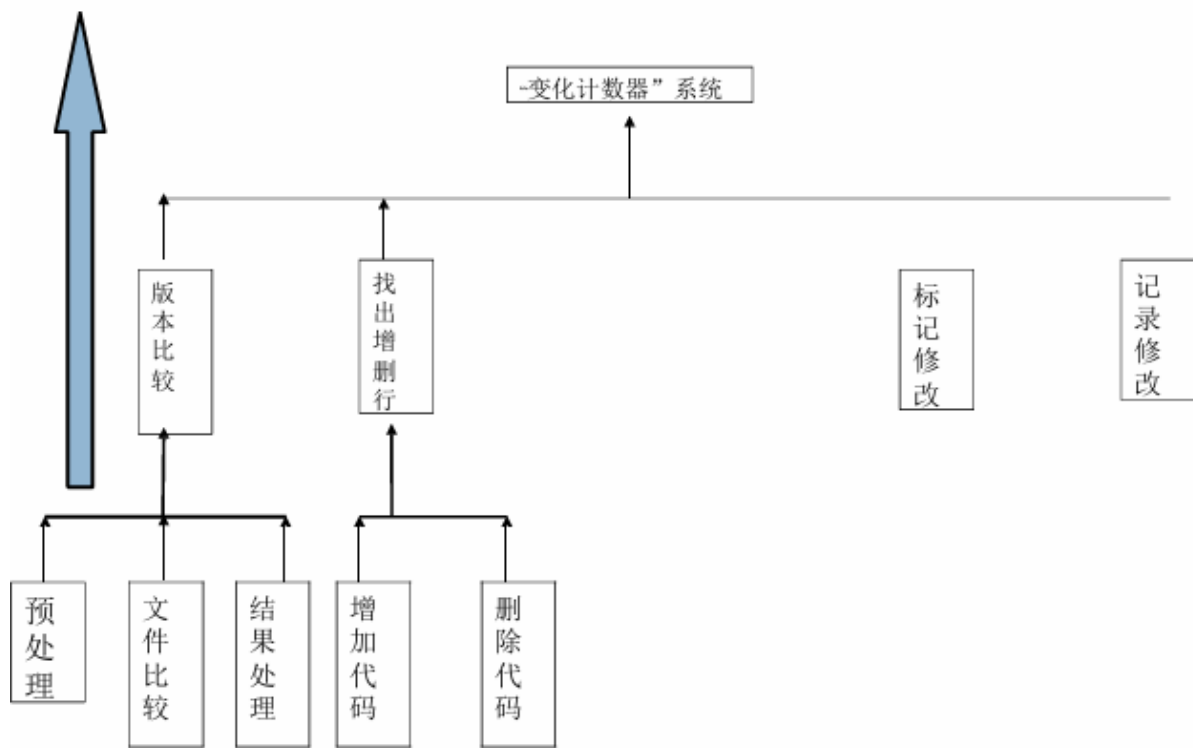
模板参照：



自上而下：



自下而上:



[很可能简答]WBS任务分解建议:

最低层是可控的和可管理的，但是不必要的过细；

每个Workpackage必须有一个提交物；

定义任务完成的标准；

有利于责任分配；

推荐任务分解到40小时以内，敏捷项目分解到3小时；

第六章：成本计划

估算过程概念：

软件项目规模：

软件项目规模即工作量，例如：软件规划，软件管理，需求，设计，编码，测试，以及后期的维护等任务。

软件规模单位(不考)：

软件项目成本：

完成软件规模相应付出的代价。

待开发的软件项目需要的资金。

人的劳动的消耗所需要的代价 是软件产品的主要成本

货币单位

成本估算结果：

直接成本与具体项目相关的成本，例如：参与项目的人员成本

间接成本可以分摊到各个具体项目中的成本，例如：培训、房租水电、员工福利、市场费用、管理费、其他等等

传统估算方法

代码行估算法、功能点估算法、用例点估算法、类比(自顶向下)估算法、自下而上估算法、三点估算法、参数估算法、专家估算法

代码行估算法：

从软件程序量的角度定义项目规模：与具体的编程语言有关、分解足够详细、有一定的经验数据

主要优缺点：

主要优点：代码是所有软件开发项目都有的“产品”，而且很容易计算代码行数。

主要缺陷：

- 1.对代码行没有公认的可接受的标准定义
- 2.代码行数量依赖于所用的编程语言和个人的编程风格.
- 3.在项目早期,需求不稳定、设计不成熟、实现不确定 的情况下很难准确地估算代码量.
- 4.代码行强调编码的工作量,只是项目实现阶段的一部分

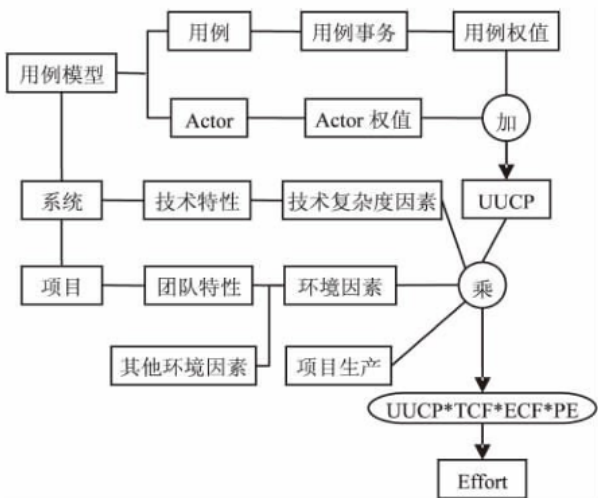
功能点估算：

与实现的语言和技术没有关系；用系统的功能数量来测量其规模；通过评估、加权、量化得出功能点

Albrecht功能点估算：

1979年, Alan Albrecht 提出;
也称为IFPUG(国际功能点用户组织)功能点;
适用于信息系统

用例点估算模型：



基本步骤：

- 1.计算未调整的角色权值UAW;
- 2.计算未调整的用例权值UUCW;
- 3.计算未调整的用例点UUCP;
- 4.计算技术和环境因子TEF;

- 5.计算调整的用例点UCP;
- 6.计算工作量(man - hours)。

类比估算

定义:

估算人员根据以往的完成类似项目所消耗的总成本（或工作量），来推算将要开发的软件的总成本（或工作量）。是一种自上而下的估算形式

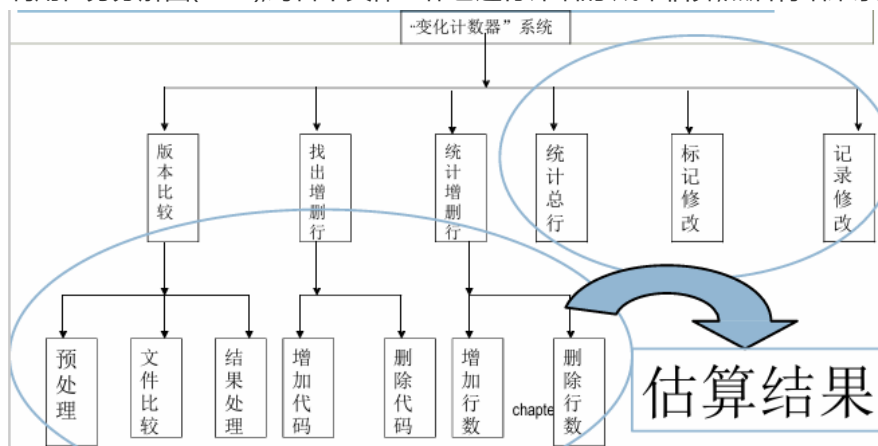
使用情况:

有类似的历史项目数据；信息不足（例如市场招标）的时候；要求不是非常精确估算的时候

自下而上估算

定义:

利用任务分解图(WBS),对各个具体工作包进行详细的 成本估算,然后将结果累加起来得出项目总成本。



特点:

相对比较准确，它的准确度来源于每个任务 的估算情况；花费时间

三点估算:

基于任务成本的三种估算值来计算预期成本的方法。

三种估算值:

最可能成本（CM）:比较现实的估算成本。

最乐观成本（CO）:最好情况所得到的估算成本。

最悲观成本（CP）:最差情况所得到的估算成本。

参数估算法

定义:

通过项目数据,进行回归分析,得出回归模型；通过参数模型估算(规模)成本的方法。

==[选择/简答、30%考COCOMO模型]面向LOC驱动的参数模型==

Walston-Felix(IBM)、Balley-Basili、COCOMO、Doty

面向FP驱动的参数模型:

Albrecht and Gaffney、Matson,Barnett

整体公式:

$$E = a + b * S^c * M$$

- E:以人月表示的工作量
- a,b,c:经验导出的系数
- M: 调节因子
- S:主要的输入参数(通常是LOC,FP等)

[看看就行]Walston-Felix模型:

1977年, IBM的Walston和Felix提出了如下的估算公式

- $E = 5.2 \times (KLOC)^{0.91}$, KLOC是源代码行数, E是工作量(以PM计)
- $D = 4.1 \times (KLOC)^{0.36}$, D是项目持续时间(以月计)
- $S = 0.54 \times E^{0.6}$, S是人员需要量(以人计)
- $DOC = 49 \times (KLOC)^{1.01}$, DOC是文档数量(以页计)

Walston-Felix模型举例:

采用java 完成项目, 估计有366功能点, 则

- $L = 366 \times 46 = 16386 \text{行} = 16.386 \text{KLOC}$
- $E = 5.2 \times (KLOC)^{0.91} = 5.2 \times 16.386^{0.91} = 66 \text{人月}$
- $DOC = 49 \times (KLOC)^{1.01} = 49 \times 16.386^{1.01} = 826 \text{页}$

==COCOMO模型: ==

结构化成本模型; 是目前应用最广泛的参数型软件成本估计模型; 由Barry Boehm团队开发的

COCOMO基本原理:

将开发所需要的工作量表示为KLOC软件规模和一系列成本因子的函数,基本估算公式:

$$PM = A \times S^E \times \prod_{i=1}^n EM_i$$

A:可以校准的常量; S为KLOC软件规模; E为规模的指数,说明不同规模软件具有的相对规模经济和不经济性; EM为工作量乘数,反映某个项目特征对完成项目开发所需工作量的影响程度;n为描述软件项目特征的成本驱动因子的个数

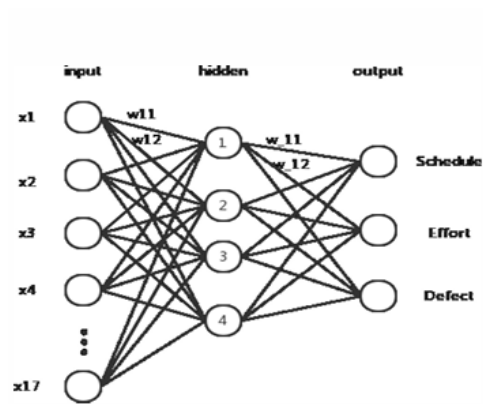
COCOMO 81项目类型:

- 有机:Organic,
 - 各类应用程序, 例如数据处理、科学计算等
 - 受硬件的约束比较小, 程序的规模不是很大
- 嵌入式:Embedded

- 系统程序，例如实时处理、控制程序等
- 紧密联系的硬件、软件和操作的限制条件下运行，软件规模任意
- 半嵌入式:Semidetached
 - 各类实用程序，介于上述两种软件之间，例如编译器（程序）
 - 规模和复杂度都属于中等或者更高

参数模型：

采用三层的BP神经网络建模：



隐含层节点 i 的输入为⁴

$$h_i = \sum w_{ij}x_j + b_i$$

隐含层节点 i 的输出为⁴

$$y_i = \theta(h_i) = \theta(\sum w_{ij}x_j + b_i)$$

隐含层的所有节点的输出就是输出层节点的输入信号，类似地可得输出层节点 k 的输入为⁴

$$out_in_k = \sum w_{ki}y_i + a_k = \sum w_{ki}\theta(\sum w_{ij}x_j + b_i) + a_k$$

输出层节点 k 的输出为⁴

$$o_k = \beta(out_in_k) = \beta(\sum w_{ki}y_i + a_k) = \beta(\sum w_{ki}\theta(\sum w_{ij}x_j + b_i) + a_k)$$

模型建立：

$$\Delta w_{ki} = \eta \cdot e_k \cdot \beta' \cdot y_i$$

$$\Delta a_k = \eta \cdot e_k \cdot \beta'$$

$$\Delta w_{ij} = \eta \cdot \beta' \cdot \theta' \cdot x_j \cdot e_k \cdot w_{ki}$$

$$\Delta b_i = \eta \cdot \beta' \cdot \theta' \cdot e_k \cdot w_{ki}$$

分析结果：

- 隐层节点数目为4，
- 网络学习率为0.54，
- 隐层激活函数为logsig，
- 输出层激活函数为purelin。
- 确定动量因子mc为0.9

参数估算法：

使用条件：具有良好的项目数据为基础；存在成熟的项目估算模型。

特点：比较简单，而且也比较准确；如果模型选择不当或者数据不准，也会导致偏差

专家估算法：

由多位专家进行成本估算，一个专家可能会有偏见，最好由多位专家进行估算，取得多个估算值，最后得出综合的估算值。

专家估算法-Deiphi:

1. 组织者确定专家，这些专家互相不见面
2. 组织者发给每位专家一份软件规格说明
3. 专家以无记名对该软件给出3个规模的估算值
 - ① 最小 a_i
 - ② 最可能的 m_i
 - ③ 最大 b_i
4. 组织者计算每位专家的 $E_i = (a_i + 4m_i + b_i) / 6$
5. 最终可以获得一个多数专家共识的软件规模： $E = E_1 + E_2 + \dots + E_n / n$ (n :表示 n 个专家)
6. 如果各个专家的估算差异超出规定的范围（例如：15%），则需重复上述过程

敏捷估算方法:

敏捷估算思维:

采用轻量级估算方法快速生成高层级估算；短期规划可以进行详细的估算。

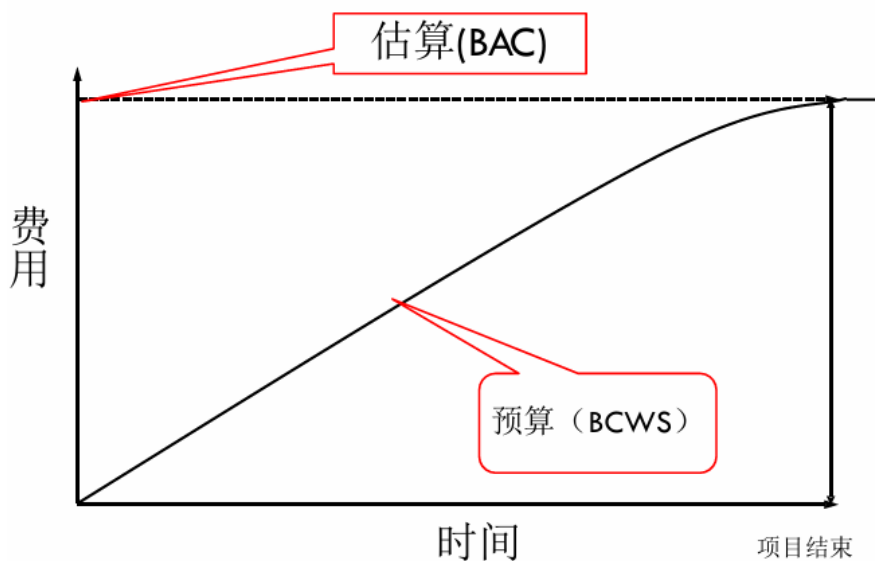
Story point估算方法:

Story point 用来度量实现一个Story需要付出的工作量的相对估算。

成本预算

成本预算是将项目的总成本按照项目的进度分摊到各个工作单元中去；
成本预算的目的是产生成本基线。

估算 (BAC) 与预算 (BCWS)



项目成本预算：

分配项目成本预算包括三种情况：

- 1.给任务分配资源成本
- 2.给任务分配固定资源成本
- 3.给任务分配固定成本

给任务分配资源成本:常规方法

与资源的费率相关：标准费率、加班费率、每次使用费率.....

分配固定资源成本：

当一个项目的资源需要固定数量的资金时，可以向任务分配固定资源成本。例如：项目中的一个兼职人员成本。

分配固定成本：

有些任务是固定成本的类型的任务，也就是说，管理者知道某项任务的成本不变，不管任务的工期有多长，或不管任务使用了那些资源。在这种情况下，管理者向任务直接分配成本。例如：某外包任务、培训任务。

第七章：进度计划

项目进度计划过程：任务定义、任务关系、历时估算、项目进度编排、项目进度确定（整一章按照该进度进行）



任务定义：确定为完成项目的各个交付成果所必须进行的诸项具体活动。

进度管理图示：

（选择分辨图）

网络图、甘特图、里程碑图、资源图、燃尽图、燃起图

网络图：

- 1、网络图是活动排序的一个输出
- 2、展示项目中各个活动以及活动之间的逻辑关系

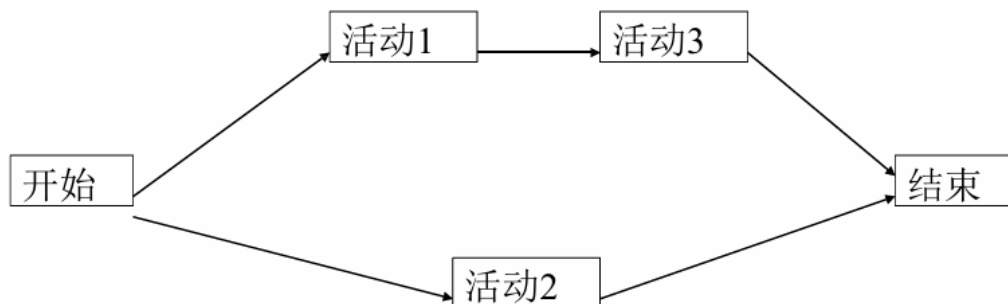
PDM (Precedence Diagramming Method)

优先图法,节点法(单代号)网络图

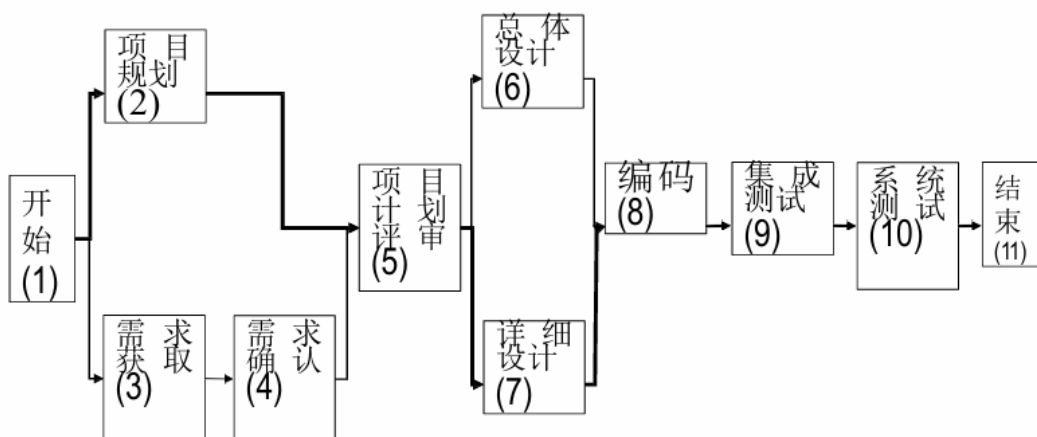
ADM (Arrow Diagramming Method)

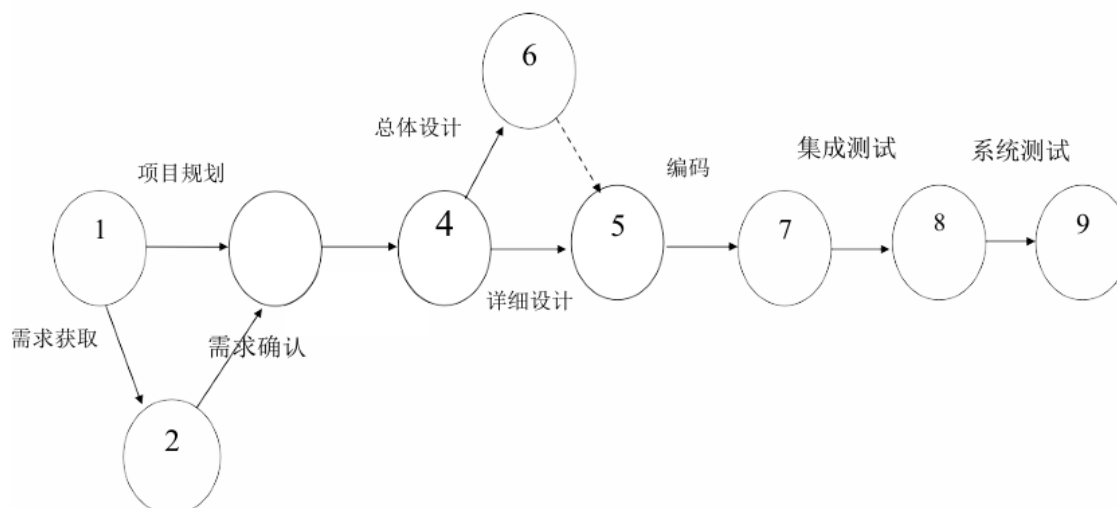
箭线法(双代号)网络图

PDM图例



PDM 图例



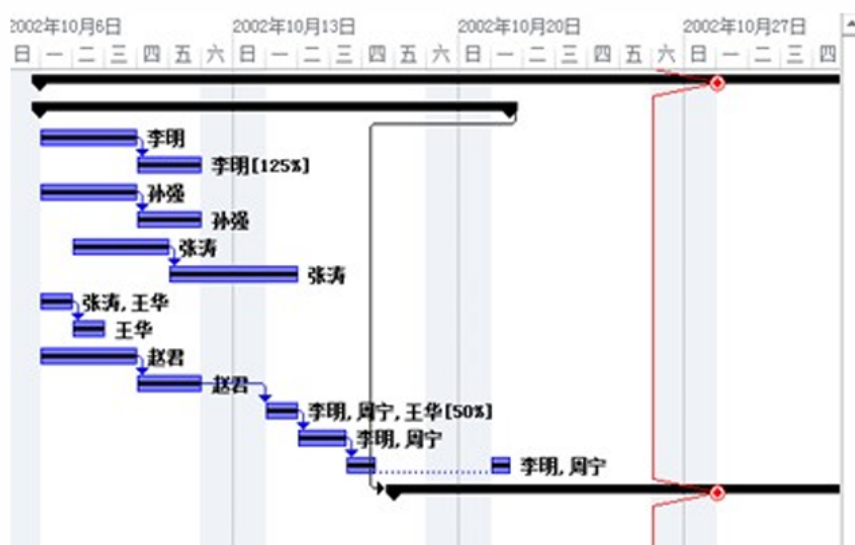


ADM(Arrow Diagramming Method)

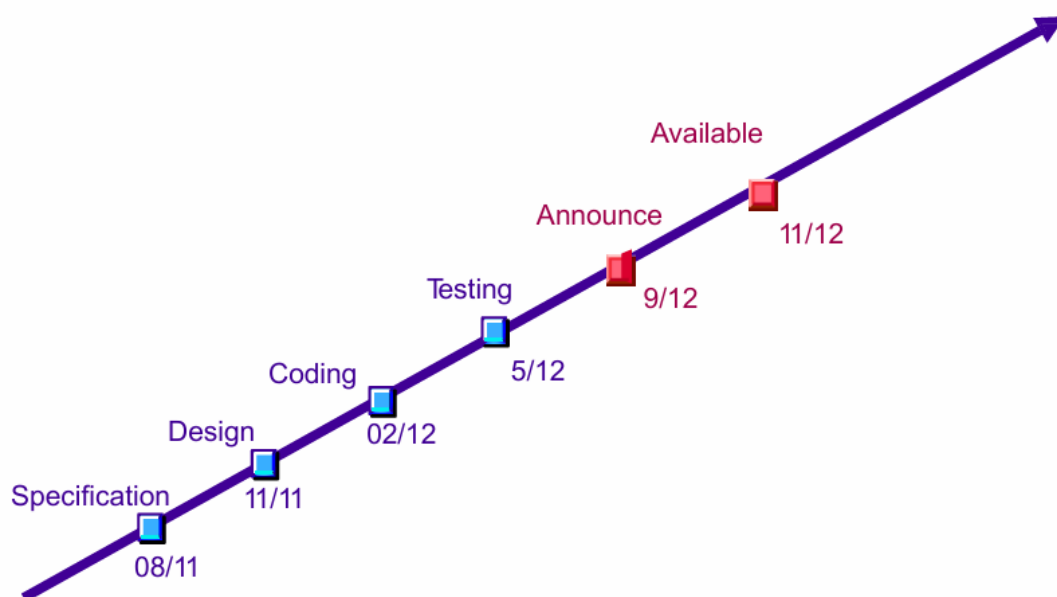


- ADM也称为双代号项目网络图,
- 在ADM网络图中,箭线表示活动(任务)
- 两个代号唯一确定一个任务
- 代号表示前一任务的结束,同时也表示后一任务的开始.

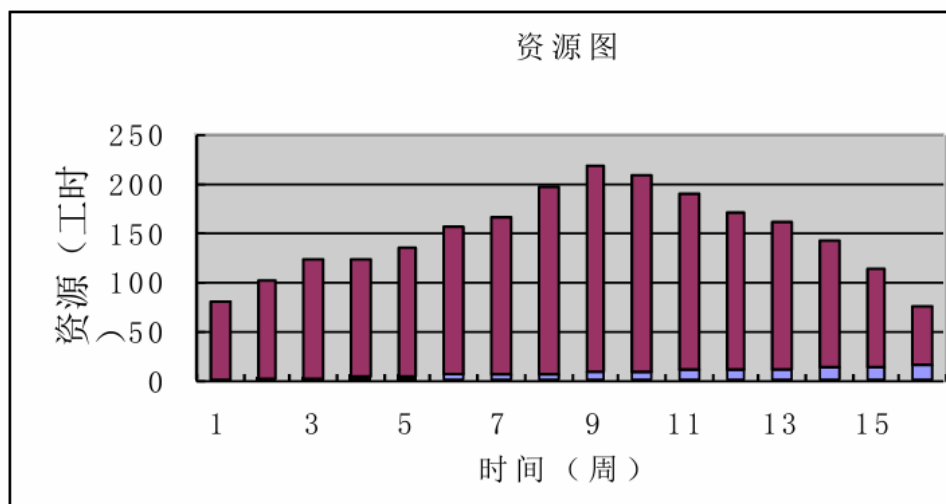
甘特图 实例



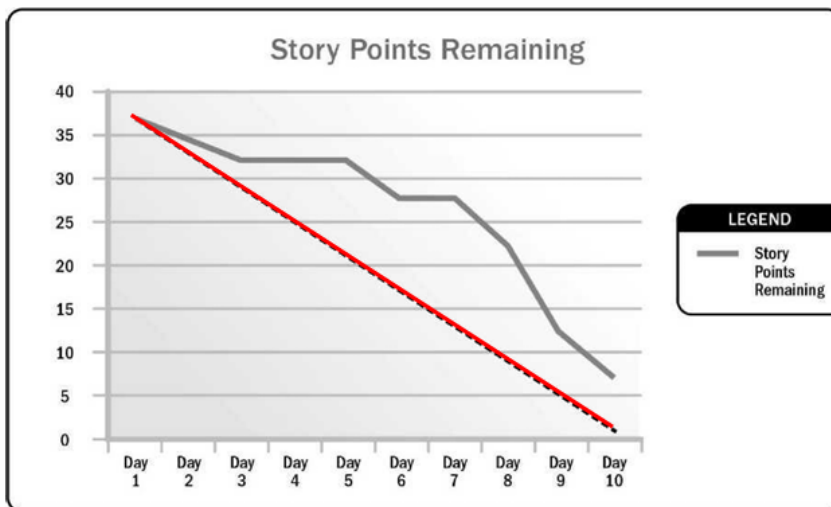
里程碑图示



资源图



燃尽图(Burndown Chart) – 进度图



燃起图(Burnup Chart) – 进度图

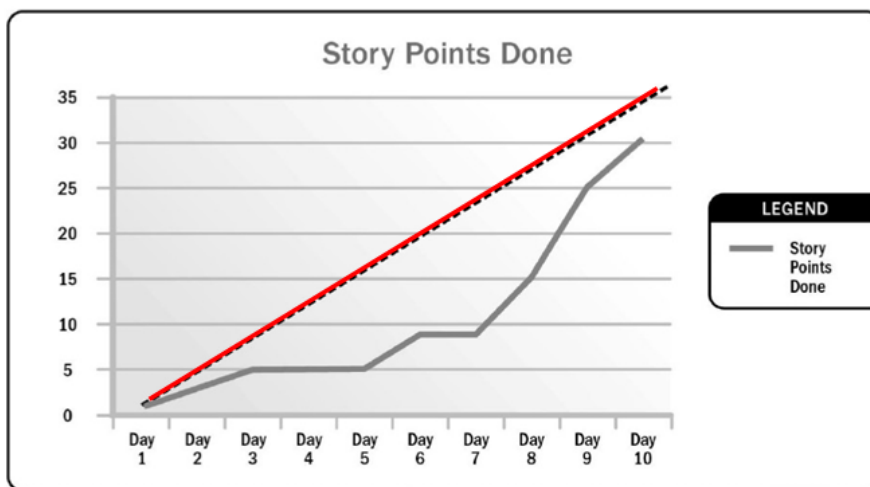


图5-2显示已完成故事点的燃起图

历时估算：估计任务、路径、项目的持续时间

历时估算的基本方法-传统：

定额估算法，经验导出模型，CPM(关键路径法估计)，PERT（工程评估评审技术），预留分析
其他（Jones的一阶估算准则，类比估计，专家判断，基于承诺的估计）

这边我记得杨海说只需要知道关键路径计算？

- 确定项目网络图
- 每个任务有单一的历时估算
- 确定网络图中任务的逻辑关系
- 关键路径是网络图中最长的路径。
- 关键路径可以确定项目完成时间

工程评估评审技术 (PERT)



- (Program Evaluation and Review Technique) 利用网络顺序图逻辑关系
- 项目中某项单独的活动，存在很大的不确定性。
- 加权算法估算任务历时
- 利用网络图逻辑关系，确定路径、项目历时

工程评估评审技术 (PERT)-加权算法



- 选定3个估算值
 - O是最小估算值:乐观(Optimistic),
 - P是最大估算值:悲观(Pessimistic),
 - M是最大可能估算(Most Likely)。
- 采用加权平均得到期望值 $E = (O + 4m + P) / 6$,

预留分析: 应急预留、管理预留

应急预留: 包含在进度基准中的一段储备时间，用来应对已经接受的已识别风险、以应对进度方面的不

确定性。
管理预留：为管理控制的目的而特别留出的项目预算，用来应对项目范围中不可预见的风险。

Jones的一阶估算准则



- 估算项目功能点
- 从幂次表中选择合适的幂次将功能点升幂

软件类型	最优级	平均	最差级
系统软件	0.43	0.45	0.48
商业软件	0.41	0.43	0.46
封装商品软件	0.39	0.42	0.45

类比估算：以过去类似项目的实际持续时间为依据，来估计当前项目的持续时间。

专家判断：根据下面专业知识而做出的历时估算。

进度计划、有关估算、学科或应用知识

基于承诺的进度估算：

要求开发人员做出进度承诺。

不进行中间的工作量（规模）估计。

主要优点：有利于开发者对进度的关注。

历时估算的基本方法-敏捷：

开发速度稳定前，开发速度稳定后。

开发速度稳定前：

项目经理组织举手表决：

举拳头表示不支持，

伸五个手指表示完全支持，

伸出三个以下手指的团队成员有机会与团队讨论其反对意见，

不断进行举手表决，直到整个团队达成共识（所有人都伸出三个以上手指）或同意进入下一个决定

开发速度稳定后：

基于故事点生产率的估算。

基于迭代生产率的估算。

项目进度编制：

基本方法：超前（Lead）与滞后（Lag）

关键路径法

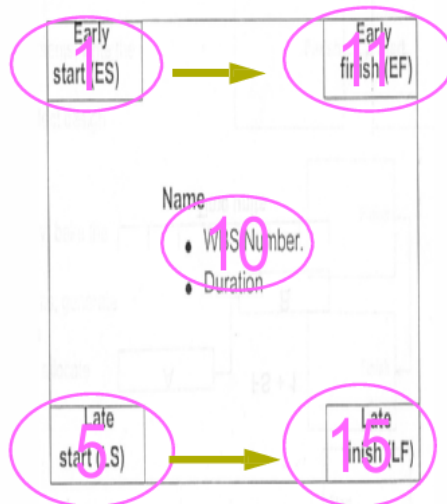
时间压缩法

资源优化

敏捷计划

关键路径基本概念：最早开始时间，最晚开始时间，最早完成时间，最晚完成时间，总浮动，自由浮动

ES、EF、LS、LF



浮动时间：浮动时间是一个任务的机动性，它是一个任务在不影响其他任务或者项目完成的情况下可以延迟的时间量。（具体还是按照题目来）

总浮动与自由浮动：

总浮动 (Total Float) :

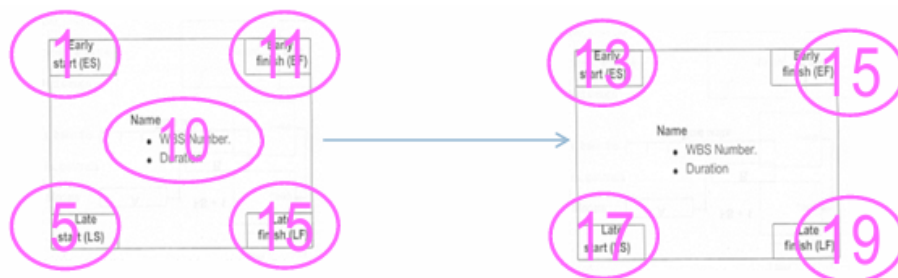
在不影响项目最早完成时间的前提下，一个任务可以延迟的时间。

自由浮动(Free Float):

在不影响后置任务最早开始时间的前提下，一个任务可以延迟的时间

TF=4

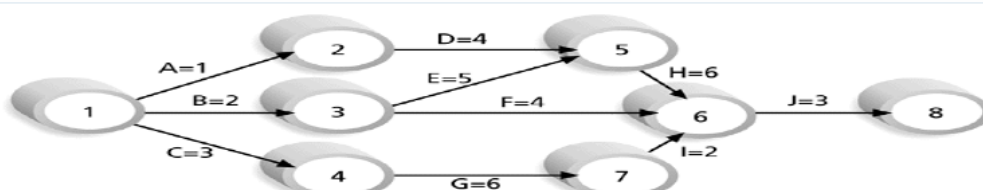
FF=2



关键路径 (Critical Path)



- ❑ 网络图中最长的路径
- ❑ 关键路径是决定项目完成的最短时间。
- ❑ 时间浮动为0 (Float=0) 的路径
- ❑ 关键路径上任何活动延迟, 都会导致整个项目完成时间的延迟
- ❑ 关键路径可能不止一条



Note: Assume all durations are in days.

Path 1: A-D-H-J Length = 1+4+6+3 = 14 days

Path 2: B-E-H-J Length = 2+5+6+3 = 16 days

Path 3: B-F-I-J Length = 2+4+3 = 9 days

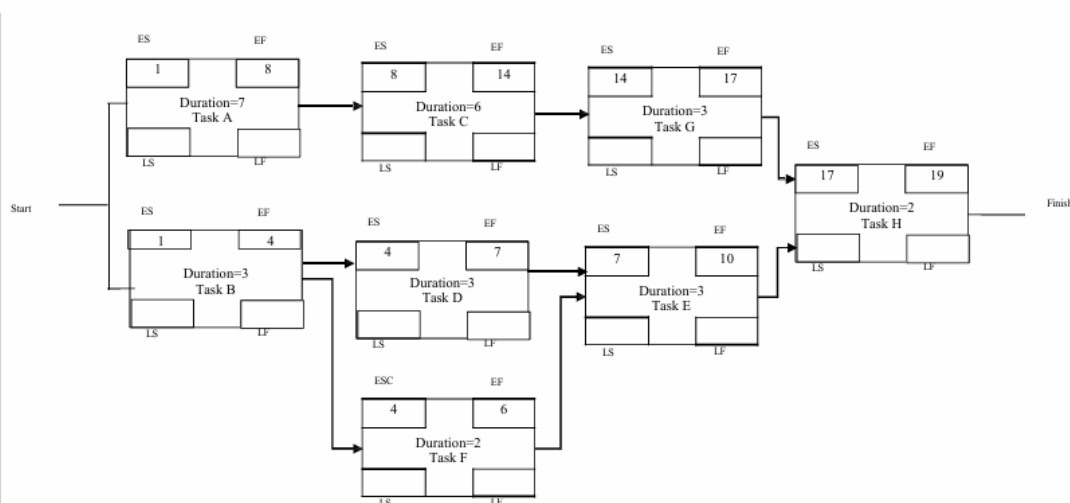
Path 4: C-G-I-J Length = 3+6+2+3 = 14 days

Since the critical path is the longest path through the network diagram, Path 2, B-E-H-J, is the critical path for Project X.

这边具体解题看ppt.

正推法: 按照时间顺序计算最早开始时间和最早完成时间的方法, 称为正推法

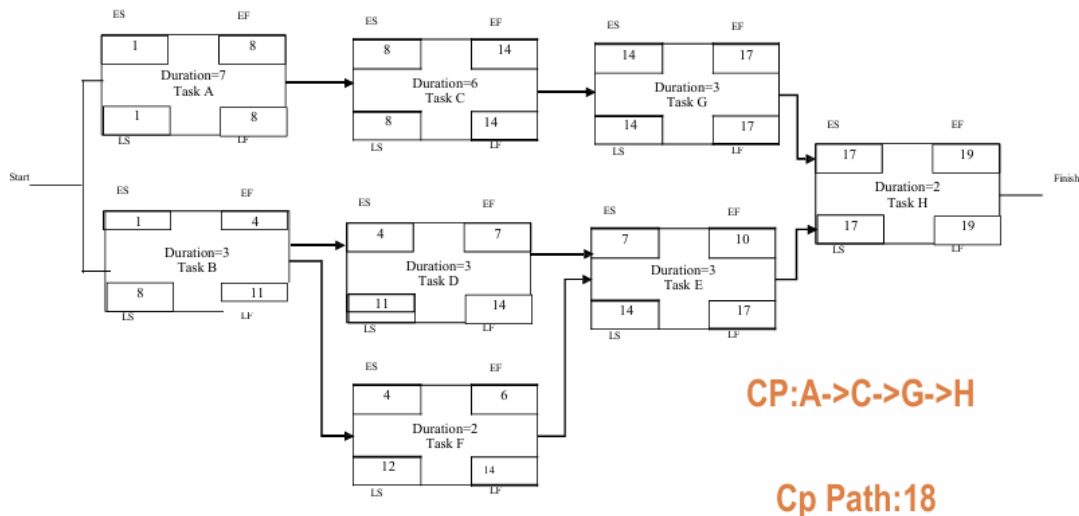
- 确定项目的开始时间, 网络图中第一个任务的最早开始时间是项目的开始时间;
- $ES + \text{Duration} = EF$
- $EF + \text{Lag} = ES$ (s), 当一个任务有多个前置任务时, 选择前置任务中最大的EF加上Lag作为其ES。
- 依次类推, 从左到右, 从上到下, 计算每个路径的所有任务的ES和最早完成时间EF



当一个任务有多个前置任务时, 选择前置任务中最大的EF加上Lag作为其ES。

逆推法: 按照你时间顺序计算最晚开始时间和最晚结束时间的方法, 称为逆推法。

- 首先确定项目的结束时间, 网络图中最后一个任务最晚完成时间是项目的结束时间;
- $LF - Duration = LS$
- $LS - Lag = LF(p)$, 当一个任务有多个后置任务时, 选择其后置任务中最小LS减Lag作为其LF
- 依次类推, 从右到左, 从上到下, 计算每个任务的最晚开始时间LS和最晚结束时间LF



当一个任务有多个后置任务时, 选择其后置任务中最小LS减Lag作为其LF。

时间压缩法: 时间压缩法是在不改变项目范围的前提下缩短项目工期的方法。

分为: **应急法——赶工**

平行作业法——快速跟进

应急法——赶工:

在最小相关成本增加的条件下, 压缩关键路径上的关键活动历时的方法

赶工也称为时间-成本平衡方法

平行作业法——快速跟进:

改变活动间的逻辑关系, 并行开展某些活动。

资源优化:

1、根据资源供需情况, 调整活动的开始和完成日期。

2、资源优化配置, 形成最有效的利用资源

使资源闲置的时间最小化

尽量避免超出资源能力

敏捷计划:

Release planning- 发布计划- 远期计划-粗计划

Iteration planning- 迭代计划- 近期计划-细计划

计划优化调整:

1、调整资源, 解决资源冲突

2、调整进度, 优化项目, 缩短工期

3、调整项目成本预算, 以便减少项目费用。

例如网络分析技术

第八章：软件项目质量计划

本章要点：

一 软件质量基本概念

二 软件质量活动

三 敏捷质量活动

四 软件质量计划

五 案例分析

软件质量基本概念：

软件质量

质量模型

质量的形成

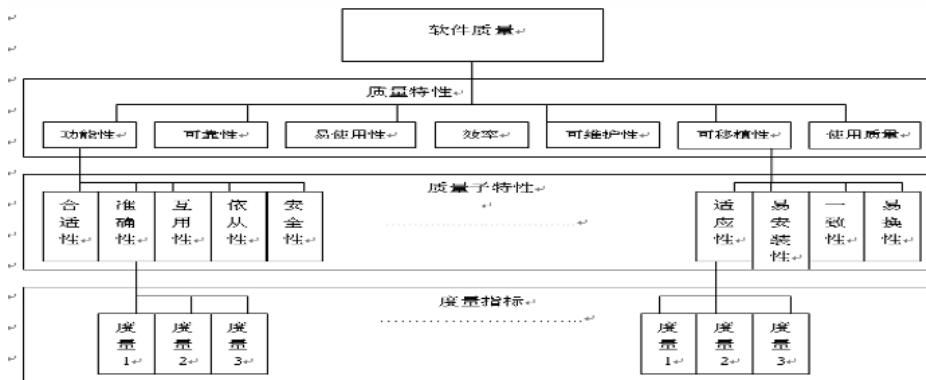
质量：质量是满足要求的程度，包括符合规定的要求和满足顾客隐含要求。

软件质量：软件质量是软件满足明确说明或者隐含的需求的程度

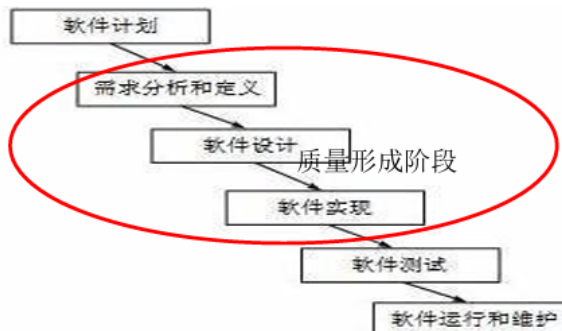
软件质量模型：人们通常把影响软件质量的特性用软件质量模型来描述。

- 1976年 Boehm质量模型
- 1979年 McCall质量模型
- 1985年 ISO质量模型

复习ppt上还有张图是ISO/IEC9126模型



质量的形成：质量形成于产品或者服务的开发过程中，而不是事后的检查（测试）把关等。



质量成本（CoQ）：

预防成本：前期质量成本

缺陷成本：后期质量成本

软件质量管理活动：

软件质量保证，软件质量控制

敏捷项目的质量规划特征：

全程质量审查

早发现问题,多版本提交

不断进行质量方法评估和改进

敏捷项目的质量活动：

结对编程(Pair Programming)

TDD(Test Driven Development)

持续集成与测试

不同层面测试

验收测试驱动开发 (ATDD)

迭代评审

迭代回顾会议

Refactor

测试驱动开发 (TDD) :

测试驱动开发的基本思想就是在开发功能代码之前，先编写 测试代码，然后编写相关的代码满足这些测试用例。

持续集成与测试:

频繁地将工作集成到整体中，然后再进行重新测试， 以确定整个产品仍然按照预期工作。

不同层面自动化测试:

单元测试

集成测试

系统级测试

冒烟测试

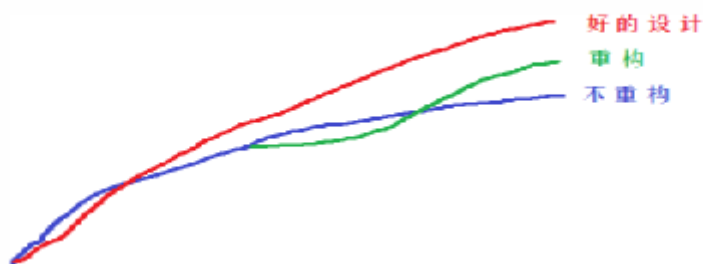
回归测试

验收测试驱动开发 (ATDD): 与客户一起讨论验收标准，创建测试用例，据此驱动代码编写，进行自动化测试，满足验收标准。

重构:

简单设计之后编写可以运行的代码

版本运行之后逐步完善代码和设计



软件质量计划:

- 1、确定项目应达到的质量标准（目标）。
- 2、决定如何满足质量标准的计划安排和方法

质量计划方法

试验设计

基准对照

质量成本分析

数据图形分析: 流程图方法，因果分析图，思维导图

第九章： 软件项目配置管理计划

注：从本章开始，不为考核重点，只会出小题不会出大题。

配置管理定义:

记录软件产品的演化过程

得到精确的产品配置。

最终保证软件产品的**完整性、一致性、追溯性、可控性**

配置管理的主要功能:

版本管理、变更管理、其他

基线定义：

基线提供了软件生存期中各个开发阶段的一个**特定点**, 标志开发过程一个阶段的结束,或者里程碑。
一个(些)配置项形成并通过审核, 即形成基线

基线修改需要按照正式的程序执行

配置基本管理过程

1、配置项标识、跟踪

将项目中需要进行控制的部分拆分成SCI

2、配置管理环境建立

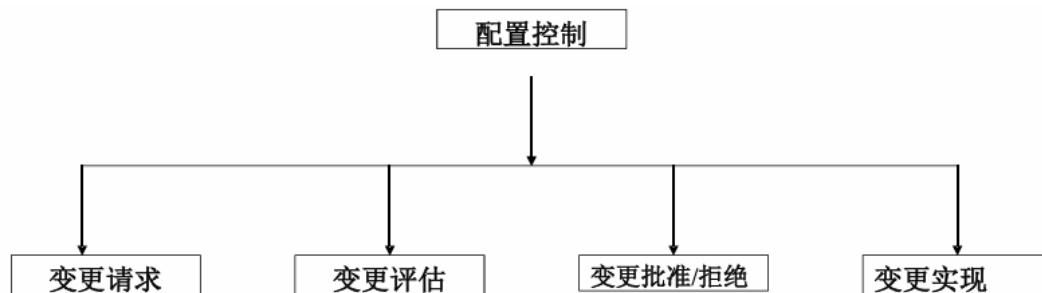
建立配置管理库

软件配置管理库是用来存储**所有基线配置项**及相关文件的等内容的系统, 是在软件产品的**整个生存期**中建立和维护软件产品完整性的主要手段。

3、基线变更管理过程

基线修改应受到控制, 这种变化要经SCCB授权, 按程序进行控制并记录基线修改的过程。

基线变更系统:



4、配置管理审计

配置管理过程审计

基线审计

5、配置状态统计

例如：

- ✓ 被批准的配置项
- ✓ 变更请求的数量
- ✓ 配置项的所有请求的变化状态
- ✓ 配置项所有被批准的变更实现状态
- ✓ 配置管理系统以及SCCB在运作中发生异常的次数
- 等等

6、配置管理计划大纲举例

- 人员职责（确定SCCB等）
- 配置项定义
- 基线定义
- 版本控制（说明配置管理工具）
- 定义变更控制系统

敏捷配置管理

敏捷的一个重要特征是持续交付, 因此, 配置管理是重要的要素

敏捷需要全面配置管理

全面配置管理的基本要求

代码和编译构建产物的配置管理
应用的配置管理
环境的配置管理
代码和编译构建产物的配置管理
制定有效的分支管理策略
配置管理工具
制定有效的分支管理策略
基于分支的开发
基于主干的开发

第十章：软件项目团队计划

组织结构的主要类型：
职能型，项目型，矩阵型

选择合适的项目角色

例如：

1)

项目经理

2)

系统分析员

3)

系统设计员

4)

数据库管理员

5)

支持工程师

6)

程序员

7)

质量保证人员

8)

配置管理人员

9)

业务专家（用户）

10)

测试人员

等等

人员职责计划：
责任分配矩阵(RAM)
组织分解结构(OBS)
其它,例如文本型

干系人：
干系人（stakeholder）是能影响项目决策、活动或者结果的个人、群体或者组织，以及会受到或者自认为会受到项目决策、活动或者结果影响的个人、群体或者组织。

项目沟通的方式：
1.书面沟通和口头沟通
2.语言沟通和非语言沟通
3.正式沟通和非正式沟通
4.单向沟通和双向沟通
5.网络沟通

项目沟通活动的分类：
内部与外部
正式和非正式
渠道（上级沟通,下级沟通,横向沟通）
书面与口头

项目沟通计划：

沟通计划是确定谁需要信息，需要什么信息，何时需要信息，以及如何将信息分发给他们。

敏捷的角色：

敏捷的角色



敏捷的角色

- 产品负责人(Product owner)
- 团队促进者(Team facilitator)
- 跨职能团队成员(Cross-functional team member)

Scrum 角色

- Product Owner （产品负责人）
- Scrum Master （ Scrum主管）
- 开发团队

敏捷团队



- 最有效的敏捷团队往往由三到九个成员组成。（黄金人数5-9人）
- 理想情况下，敏捷团队应该集中在一个工作场所工作。
- 团队成员100%为专职成员，协同工作，
- 敏捷鼓励自我管理团队。

仆人式领导：

仆人式领导是通过对团队服务来领导团队的

注重理解和关注团队成员的需要和发展

仆人式领导为团队赋权

旨在使团队尽可能达到最高绩效。

敏捷方法提倡高度透明

将项目信息发布到公告空间

第十一章：软件项目风险计划

风险定义：

风险是对潜在的、未来可能发生损害的一种度量，如果风险确实发生了，则会对项目产生有害的或者负面的影响。

软件风险对软件开发过程及软件产品本身可能造成 的伤害或损失。

风险类型：

预测角度：已知风险、可预测风险、不可预测风险

范围角度：商业风险、管理风险、人员风险、技术风险、开发环境风险、客户风险、过程风险、产品规模风险等。

风险三要素：

风险事件、事件概率、事件影响

风险管理的四个过程：

风险识别、风险评估、风险规划、风险控制

风险识别是识别风险事件,系统化地确定对项目计划的威胁, 识别已知和可预测的风险。

对风险事件发生概率的评估, 对项目风险影响的评估, 给出项目风险排序。

针对风险分析的结果, 制定一定的行动和策略来对付、减少、以至于消灭风险事件造成的影响

风险控制是在项目执行过程中实施和监控风险计划, 同时, 不断进行风险识别、风险分析、风险规划的过程。

风险识别方法:

德尔菲方法

头脑风暴法

情景分析法

利用风险条目检查表

风险评估



分析

- 风险发生的概率 (P)
- 风险对项目的影响 (I)
- 风险值, $R = F(P, I)$



确定优先次序

- 按风险值排序
- 确定最需要关注的TOP 风险

定性风险评估:

定性评估风险概率及后果

风险概率度量:

高、中、低

极高、高、中、低、极低

不可能, 不一定, 可能和极可能等等

风险影响度量:

高、中、低

极高、高、中、低、极低

灾难, 严重, 轻微, 可忽略等等

定量风险评估:

盈亏平衡分析

敏感性分析

模拟

决策树分析

决策树分析是一种图表分析方法

提供项目所有可供选择的行动方案, 行动方案 之间的关系, 行动方案的后果以及发生的概率

提供选择一个最佳的方案的依据

- *EMV*(损益期望值)是决策树的一种计算值
- 根据预期结果、发生的概率计算出一种期望的损益

例如：

某行动方案成功的概率是50%，收益是10

$$EMV = 10 * 50\% = 5$$

风险规划的主要策略：

- 1、回避风险
- 2、转移风险
- 3、损失控制
- 4、自留风险

回避风险



- 回避风险是对可能发生的风险尽可能的规避，采取主动放弃或者拒绝使用导致风险的方案
- 例如放弃采用新技术

转移风险



转移风险是为了避免承担风险损失，有意识将损失或与损失有关的财务后果转嫁出去的方法。

例如

- 分包
- 开脱责任合同
- 保险

损失控制

损失预防

例如：项目技术培训，预防技术失败

损失抑制

例如：项目人员储备，抑制人员流失的损失

由项目组织自己承担风险事故所致损失的措施。

敏捷项目风险应对方法

损失预防与损失抑制策略

- 跨职能项目团队(识别风险)
- 选择迭代内容(选择风险小的)
- 频繁评审增量产品
- 持续测试可以及早发现问题
- 客户参与可以减少需求变更的风险

敏捷项目存在风险：

- 没有长期计划,识别一些风险比较困难.
- 没有长期规划，存在变更

第十二章：软件项目合同计划

项目采购：为了执行项目而从项目团队外部获取产品、服务或者结果的过程，称为采购

合同：合同是具有法律效力的协议

双方自愿达成的协议

签订者具有相应的法律能力

有充分的签约理由

具有合法的目的

合同类型：

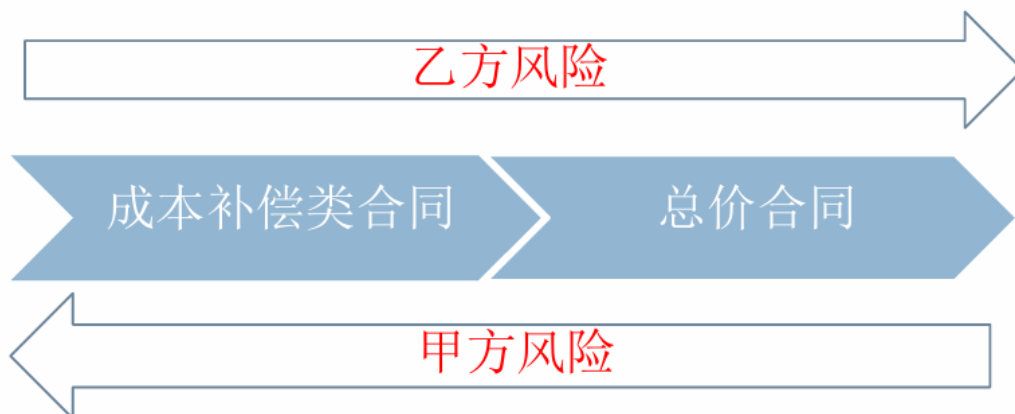
成本补偿合同，总价合同，工料合同(T&M)

成本补偿合同：向卖方支付为完成工作而发生的全部合法实际成本，外加一笔费用作为卖方的利润。
这种合同适用于：工作范围预计会在合同执行期间发生重大变更。

总价合同：总价合同为既定产品、服务或成果的采购设定一个总价。
这种合同应在已明确定义需求，且不会出现重大范围变更的情况下使用。

工料合同(T&M)合同：成本补偿合同和总价合同特点的混合型合同。
必须为每一个单位的工作量付出一定的报酬

合同类型相应的风险



合同计划：

明确如何进行委托、委托什么项目、何时进行、费用如何等，
选择需要的合同类型，采用的招标方式、合同形式等。

客户协作高于合同协商

《敏捷宣言》认为“客户协作高于合同协商”

敏捷合同策略

多层协议结构

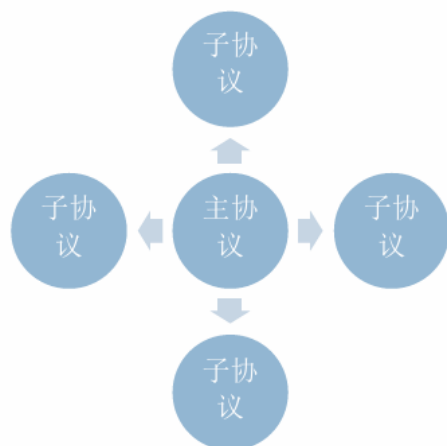
价值交付

总价增量

灵活总价方案

动态范围方案

多层协议结构



总价增量：

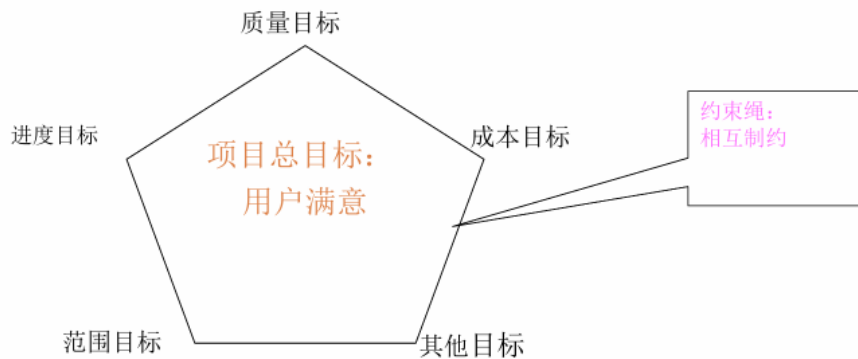
总价增量策略是无限定项目总预算,而是将范围分解 为一些微型交付成果，例如以Story为单位的交付成果,单独计算成本预算,可以很好的控制成本预算.

动态范围方案：

动态范围方案是对于具有固定预算的合同，供应商 可为客户提供在项目特定点改变项目范围的方案。

第十三章：项目集成计划执行控制

项目集成管理



项目集成管理由项目经理负责。

软件项目管理的重要四个要素

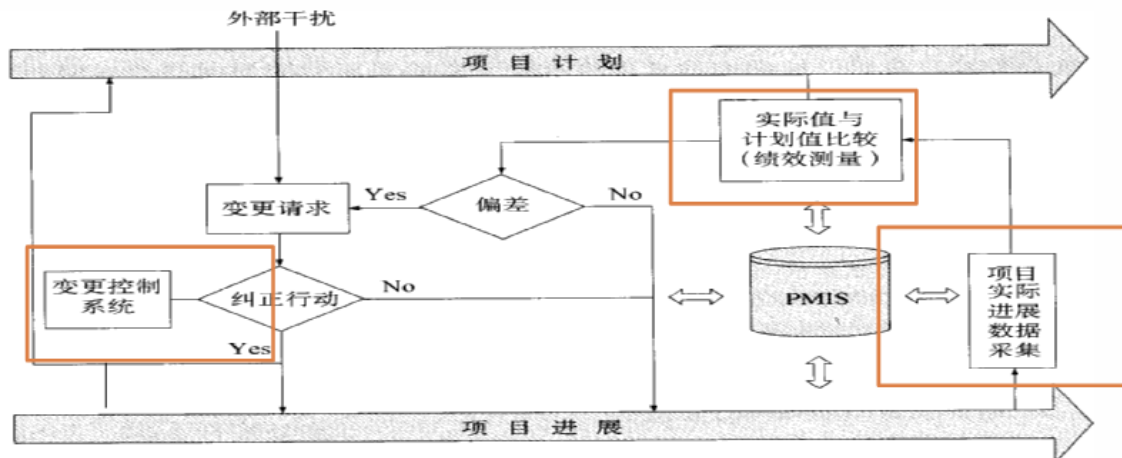
- 范围(S)
- 质量(Q)
- 进度(T)
- 成本(C)



四要素的关系

$$C=F(S,Q,T)$$

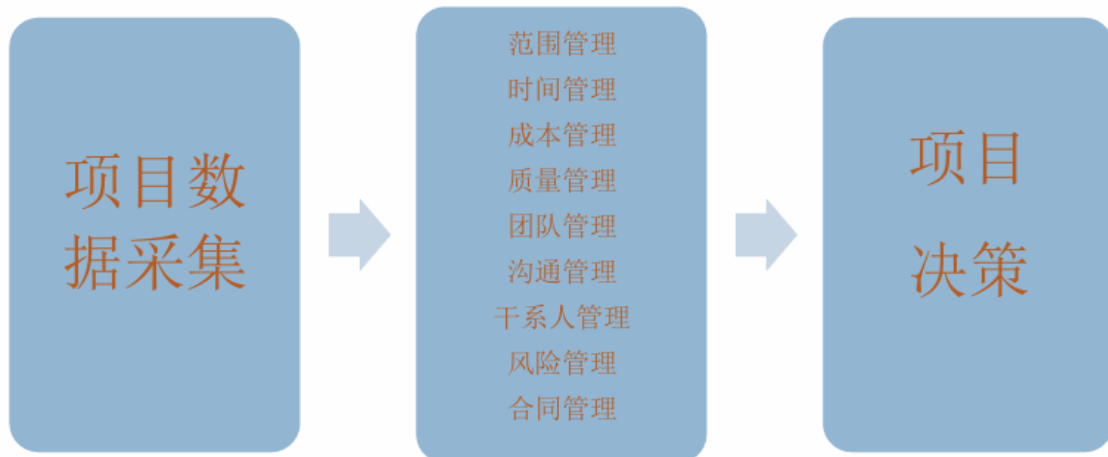
- C 与 S 成一定正比关系
- C 与 Q 成一定的正比关系
- C 与 T 成一定的反比关系



项目执行控制的步骤:

建立标准、采集项目实际数据、实际结果与计划比较、决定是否修正计划、修正计划，协调各方。

项目数据是项目执行控制的依据



项目计划修改:

计划的不合理，客观原因导致

敏捷项目的集成管理:

敏捷方法能够促进团队成员以相关领域专家的身份参与整合管理。

项目经理的关注点在于营造一个合作型的决策氛围。

对具体产品的规划和交付授权给团队来控制。

确保团队有能力应对变更。

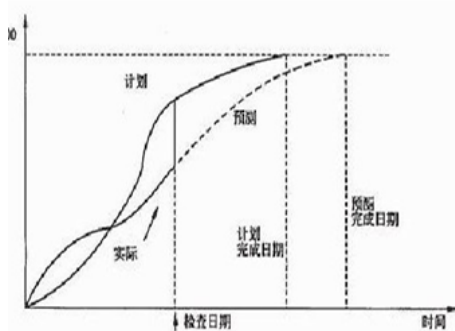
变更过程可视为一个敏捷项目。

第十四章：项目核心计划执行控制

范围控制：范围执行控制是监督项目的范围状态，管理范围基准变更的过程。

分析技术：偏差分析、趋势分析

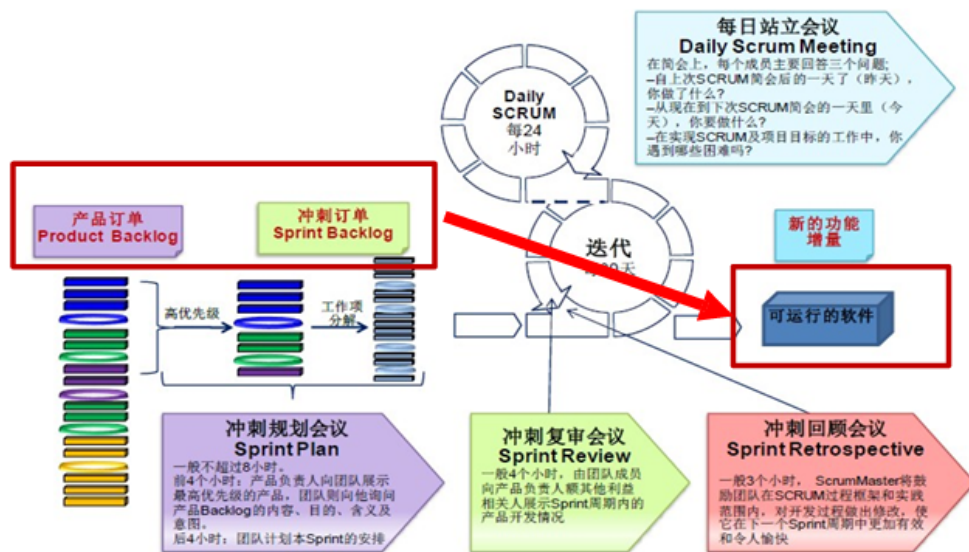
偏差分析



趋势分析



敏捷项目:需求不断被定义



Copyright@DJ 2011~2015

图3-2 Scrum方法中的主要活动和交付件

敏捷项目范围管理：

把需求列入未完项

不断构建和评审原形系统

通过发布多个版本来明确需求。

性能分析的主要技术：

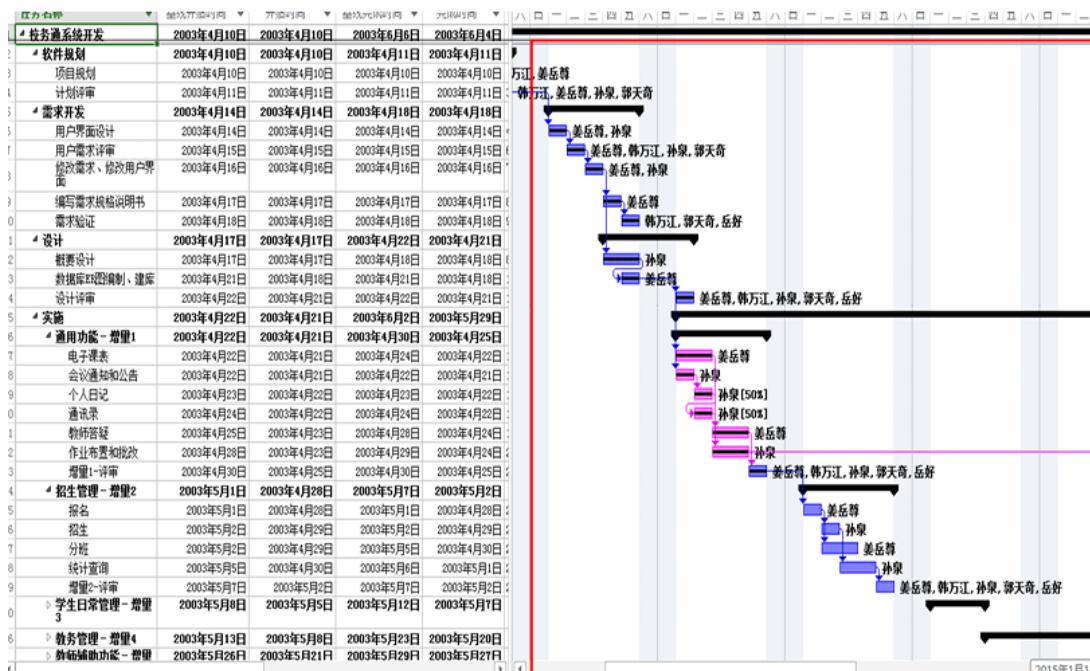
图解控制法

挣值分析法

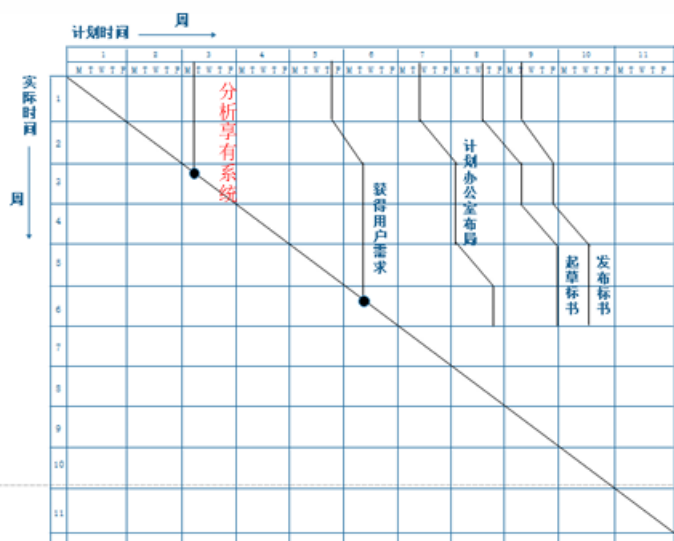
网络图分析

敏捷方法

1) 项目甘特图

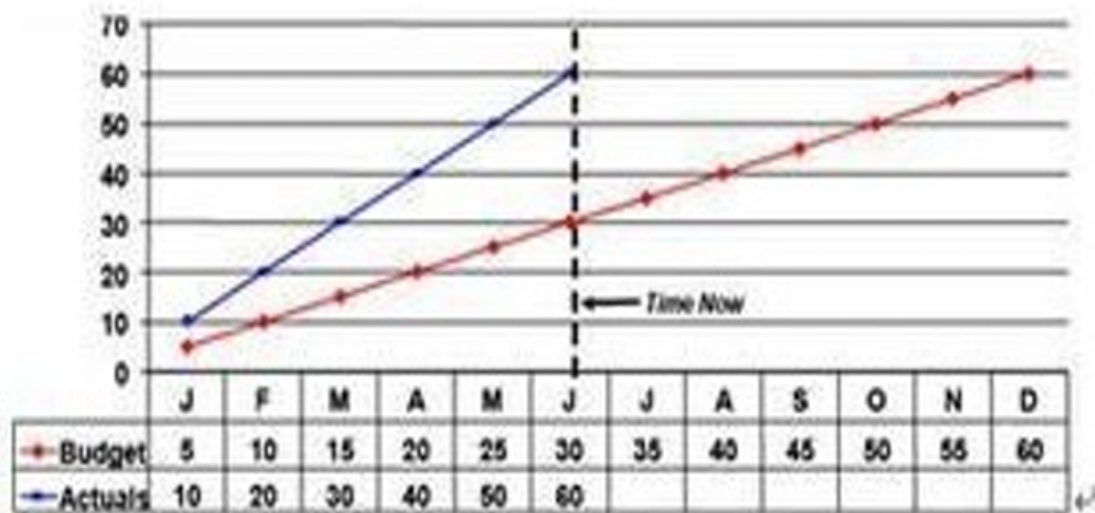


3) 时间线

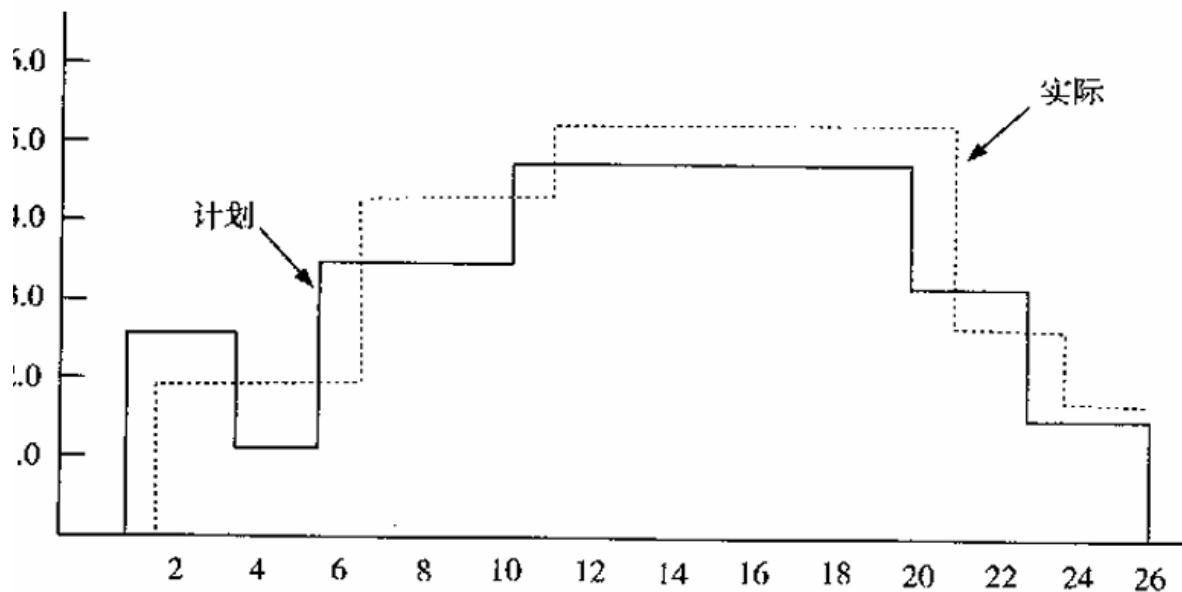


第6周的周末Brigette的时间线图

4) 费用曲线图



5) 资源图偏差

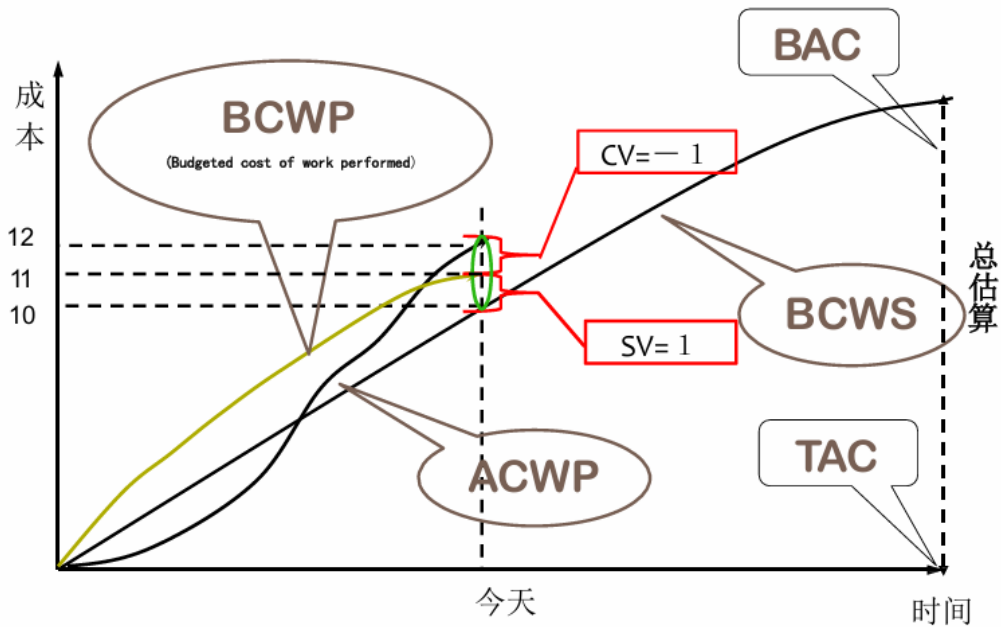


偏差分析与控制:

精确记录任务消耗的实际时间

量化任务的计划偏差

对计划偏差进行根因分析



敏捷项目进展衡量的原则：

交付价值替代预测型衡量指标

这边有两张图，分别是燃尽图和燃起图

基于流程的衡量指标：

交付周期：

交付一个工作项目花费的总时间，从项目添加到看板直至项目完成

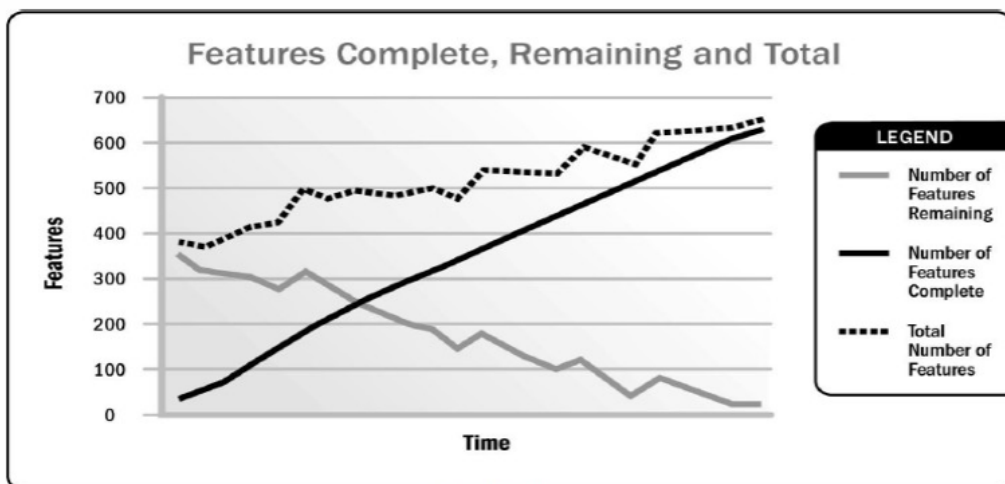
周期时间：

处理一个工作项目所需的时间

响应时间：

一个工作项目等待工作开始的时间

(三)功能燃起图/燃尽图



完成指标管理：

迭代速率：反映了一个团队在一个迭代周期内所能交付的Story 个数。

- 例如:
 - ▣ 迭代速率=50个Story points/迭代
 - ▣ 项目有500 Story points
- 则，项目大约还需要 10个迭代。

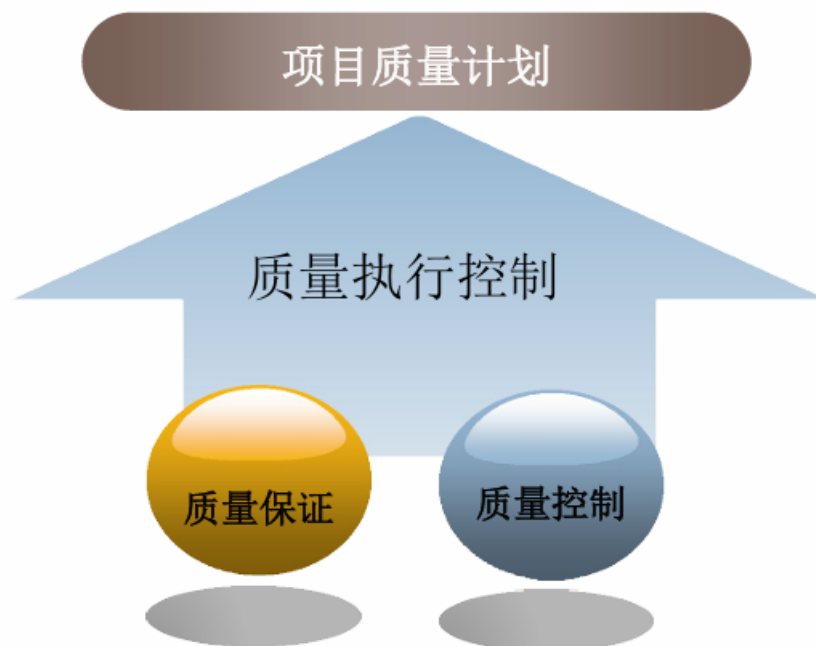
期望值管理：团队的管理者要适当控制他们的期望值的提升，因为团队的生产能力应该有它的上限。

进度变更：

通过项目进度分析,确定必要的变更

必要时,执行集成变更流程

质量执行控制



QA:质量审计 (Audit) :

过程审计，产品审计

QC:质量控制活动：



技术评审：尽早发现工作成果中的缺陷，并帮助开发员及时消除缺陷，从而有效地提高产品的质量
对等评审

QC:代码评审/走查

代码评审：由一组人通过阅读、讨论和争议对程序进行静态分析的过程。

代码走查：在代码编写阶段，开发人员自己检查自己的代码

QC:软件测试：

测试是程序的执行过程，目的在于发现错误

一个好的测试用例在于能发现至今未发现的错误；

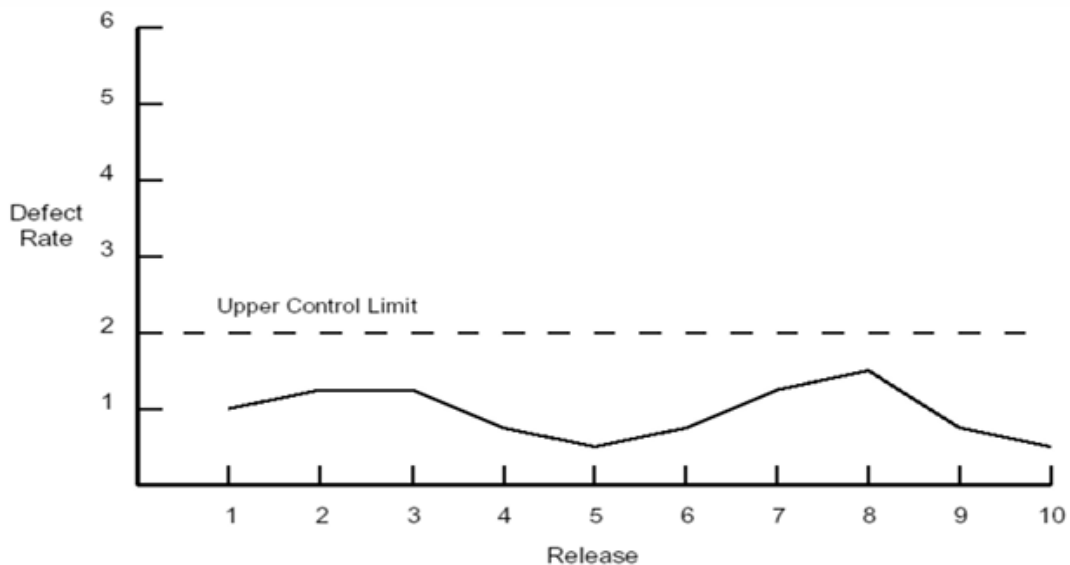
一个成功的测试是发现了至今未发现的错误的测试。

QC:返工：

返工是将有缺陷的和不合格项改造为与需求 和规格一致的行为

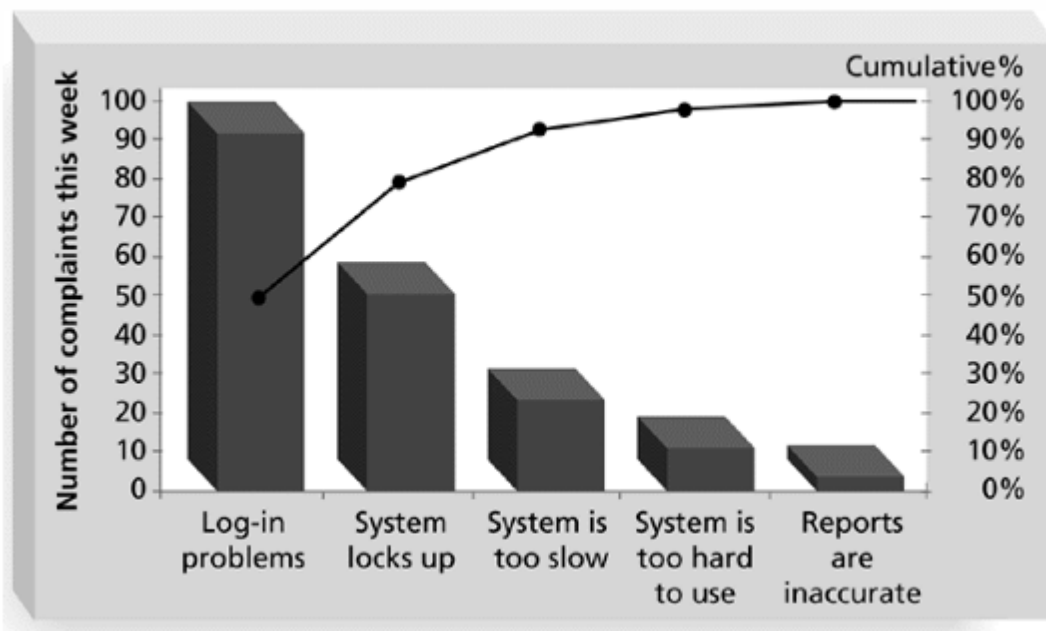
QC手段：控制图法

控制图法是一种图形的控制方法，它显示软件产品的质量随着时间变化的情况，在控制图法中标识出质量控制的偏差标准。



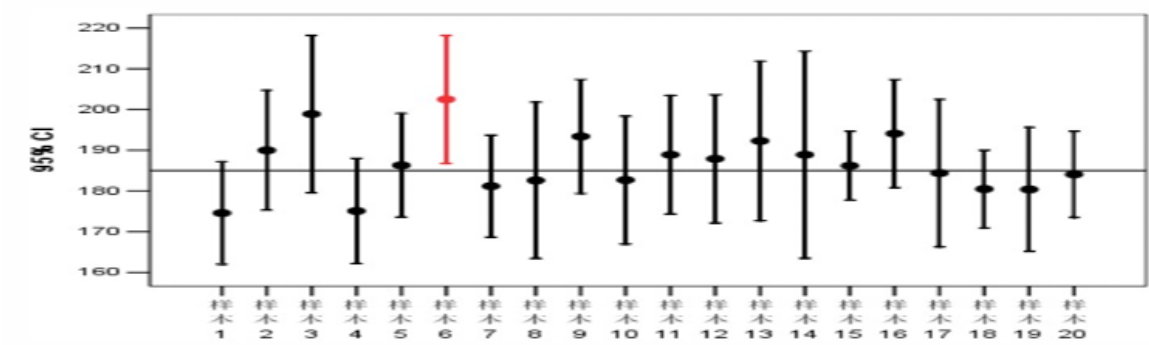
QC手段：趋势分析

趋势分析指运用数字技巧，依据过去的成果预测将 来的产品。



QC手段：抽样统计

抽样统计是根据一定的分布概率抽取部分产品进行检 查。它是以小批量的抽样为基准进行检验，以确定大量或批量产品质量的最常使用的方法。



敏捷项目质量管理过程：

- 不断进行审核
- 定期检查质量过程的效果
- 建议实施新的质量改进方法
- 回顾会议评估试验过程
- 小批量工作以便在早期发现不一致和质量问题

敏捷项目的质量活动：

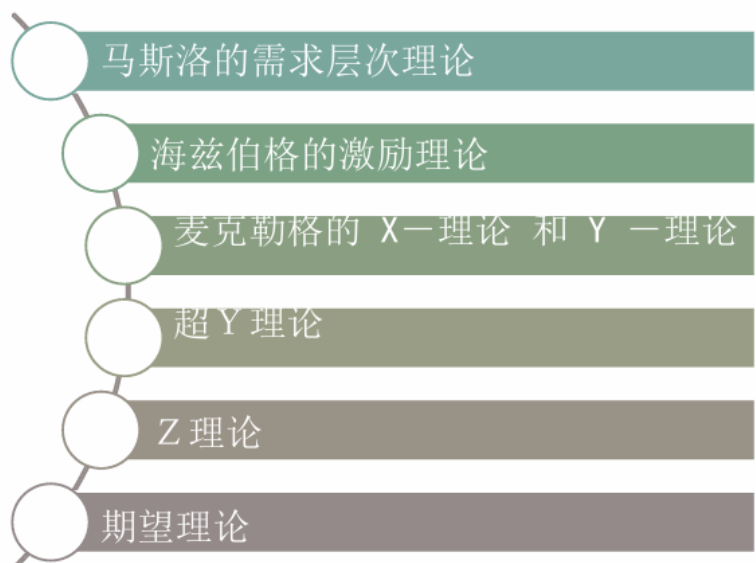
- 代码检查:PairProgramming
- TDD(Test Driven Development)
- 持续集成
- 不同层面测试
- 迭代评审
- 验收测试驱动开发 (ATDD)-Customertest
- 迭代回顾会议:Refactor

第十五章：项目辅助计划执行控制

团队计划执行控制：团队建设，项目培训，人员激励

项目培训的特点：短期、片段式、针对性强、见效快

项目成员的激励理论：



执行沟通计划：

- 书面沟通和口头沟通
- 语言沟通和非语言沟通
- 正式沟通和非正式沟通
- 单向沟通和双向沟通

网络沟通

沟通技术：

利用信息技术。

项目会议

非正式方式

其他。

风险控制过程：



合同管理：

甲方合同管理，乙方合同管理

甲方合同管理：

对采购对象的检验和验收过程

违约事件处理过程

乙方合同管理：

合同执行跟踪管理过程

合同修改控制

违约事件处理过程

产品维护过程

与客户的关系：

协作的关系：

参与接收测试

参与迭代评审会议

执行灵活的合作协议

其他

第十六章 项目结束过程

项目终止的条件：

项目计划中确认的可交付成果已经出现：

项目的目标已经成果实现

由于相关原因，项目无法继续进行：

项目已经不具备使用价值

项目无竞争力，难以生存

项目终止过程：

达到项目的完工或退出标准

完成项目合同协议

完成项目信息报告

项目结束过程：

项目验收

合同终止

项目最后评审

项目总结报告

合同终止：

合同准备、合同签署、合同管理、合同终止

项目结束前的最后任务：

项目最后评审、项目总结

项目最后评审：

是否实现项目目标.

是否遵循项目进度.

是否在预算成本内完成项目.

项目进度过程中出现问题以及解决措施是否合适 。

从该项目的实践中可以得到哪些经验和教训